



THE COMPLETE GUIDE TO

JAGUAR

Programming Language

Language reference · Standard library · Tooling
Guides · History · Development plan



JAGUAR



The Complete Guide to the Programming Language

*Language reference, standard library, C interoperability,
the JBS build system, hands-on guides, design history
and development plan.*

BASED ON JAGUARCC, JBS AND BINDGEN

Draft edition · September 2026

ABOUT THIS EDITION

This book documents the **Jaguar** programming language as it is implemented today by the JaguarCC compiler (`jcc.py`) and its companion tools: the JBS build system (`jbs.py`) and the BindGen C-header converter (`jbg.py`).

The reference chapters describe the compiler's **current implementation**, not a future specification. Some features are intentionally limited, and where the behavior is not covered by this book, the behavior of `jcc.py` is authoritative.

TYPOGRAPHY

Text is set in Caladea. Headings are set in Poppins. Code is set in DejaVu Sans Mono. This book was typeset programmatically from the Jaguar documentation.

TABLE OF CONTENTS

Contents

Preface

xii

Part I • Getting Started

1

1 Meet Jaguar	2
1.1 What is Jaguar?	2
1.2 Design philosophy	3
1.3 What Jaguar deliberately leaves out	3
1.4 The toolchain at a glance	4
2 Compiling Your First Program	6
2.1 What you need	6
2.2 Hello, Jaguar	6
2.3 Three ways to run the compiler	7
2.4 What just happened?	8
2.5 Command-line summary	9
2.6 Where to go next	9
3 A Tour of Jaguar	10
3.1 Variables and types	10
3.2 Functions	10
3.3 Control flow	11
3.4 Collections	12
3.5 Classes	12
3.6 Inheritance and virtual methods	13
3.7 Namespaces	13
3.8 The standard library	14
3.9 Talking to C	14
3.10 Building projects	14

3.11 Reflection	15
---------------------------	----

Part II • The Language 16

4 Program Structure and Lexical Elements 17

4.1 General syntax	17
4.2 Source files	17
4.3 Comments	18
4.4 Literals	18
4.5 The preprocessor	19
4.6 Top-level expressions	20

5 Types, Variables and Operators 22

5.1 Core types	22
5.2 Variables	23
5.3 Constants with const	23
5.4 Type inference with auto	24
5.5 Global variables	24
5.6 Operators	25
5.7 Casts	26
5.8 Implicit numeric conversion in calls	26

6 Strings 28

6.1 Methods at a glance	28
6.2 The string type	28
6.3 Constructing an empty string	30

7 Functions 31

7.1 Declaring and calling functions	31
7.2 Default parameters	31
7.3 Named arguments	32
7.4 Prototypes	32
7.5 Overloading	33
7.6 How a call is resolved	33
7.7 The main function	33
7.8 Returning values	34
7.9 Native functions with @extern	34

8 Control Flow and Scope	35
8.1 Conditions	35
8.2 The while loop	36
8.3 break and continue	36
8.4 The for_loop loop	37
8.5 Variable scope	37
9 Structs, Enums, Unions and Aliases	39
9.1 Structs	39
9.2 Enumerations	39
9.3 Unions	40
9.4 Forward declarations	41
9.5 Type aliases with using	41
10 Classes and Objects	42
10.1 Declaring a class	42
10.2 Access control	43
10.3 Methods	43
10.4 The this keyword	44
10.5 Constructors with constr	44
10.6 Destructors with destr	45
10.7 Inheritance	45
10.8 Virtual methods	46
10.9 const methods	46
11 Collections and Object Ownership	47
11.1 The collection types	47
11.2 Lists	47
11.3 Maps	48
11.4 Pairs	49
11.5 Dynamic lists	49
11.6 Creating collections with {}	50
11.7 Walking a list with loop_list	51
11.8 Walking a map with loop_map	51
11.9 Indexing	51
11.10 Creating objects with new	52

11.11 Containers	52
11.12 Memory management	53
11.13 Where collections can and cannot be used	53
12 Namespaces and Modules	54
12.1 Namespaces	54
12.2 Importing a namespace	55
12.3 Importing one symbol, with an alias	55
12.4 Importing a complete Jaguar file	56
13 Reflection, Attributes and Signals	57
13.1 Overview	57
13.2 Creating objects with factory:construct	58
13.3 Reading a member with GetMember	58
13.4 Testing a member with MemberExists	58
13.5 Writing a member with SetMember	59
13.6 A complete reflection example	59
13.7 Attributes	59
13.8 Signals: reacting to a variable change	60
Part III • The Standard Library	61
14 The sys Library	62
14.1 Printing	62
14.2 Console colors	63
14.3 Running a program	63
14.4 File system	63
14.5 sys at a glance	64
15 The jcc Library	65
15.1 Process control and integer helpers	65
15.2 Math	66
15.3 String functions	67
15.4 String conversions	68
15.5 Character functions	68
15.6 Files and environment	69
15.7 Reserved libc names	69

16 C Interoperability	72
16.1 The big picture	72
16.2 @extern	72
16.3 Pointer types	73
16.4 nullptr	74
16.5 C strings	74
16.6 Function-pointer types	74
16.7 ABI-preserving numeric types	75
16.8 auto and the => pointer shorthand	76
16.9 const pointer semantics	76
16.10 Global @extern variables	76
16.11 A practical C API example	77
16.12 C to Jaguar cheat sheet	77
16.13 Generated C and compiler compatibility	77
17 Jaguar Headers and BindGen	79
17.1 Jaguar headers (.jah)	79
17.2 Generating a .jah from a C header	80
17.3 How BindGen works	80
17.4 Supported C declarations	81
17.5 C strings and ABI preservation	82
17.6 Anonymous aggregate types	82
17.7 Unsupported C constructs	83
17.8 Invalid-header recovery	83
17.9 A complete example	84
18 The JBS Build System	86
18.1 Overview	86
18.2 Minimal project	86
18.3 version	87
18.4 out	87
18.5 include	87
18.6 define	88

18.7 c89	88
18.8 keep_c	89
18.9 compile	89
18.10 compile_static	90
18.11 compile_shared	90
18.12 link	90
18.13 --target	91
18.14 JBS command line	91
18.15 Toolchain	92
18.16 using expansion	92
18.17 Full project example	92
18.18 Recommended project organization	94

19 Under the Hood: How JaguarCC Works 95

19.1 From source to executable	95
19.2 What ends up in the C	96
19.3 Name mangling	96
19.4 Diagnostics and source locations	97
19.5 One unit, many files	97
19.6 Tool responsibilities	97

Part V • Guides and Cookbook 99

20 Guide: A Temperature Table 100

20.1 What we are building	100
20.2 Step 1: the conversion function	100
20.3 Step 2: the loop	101
20.4 Step 3: readable output	101
20.5 Step 4: compile and run	101
20.6 What you practiced	102

21 Guide: Modeling a Game with Classes 103

21.1 Design first	103
21.2 Step 1: the player	103
21.3 Step 2: a base class with a virtual method	104
21.4 Step 3: a derived class	104

21.5 Step 4: the fight	105
21.6 What you practiced	105
22 Guide: A Scoreboard with Collections	106
22.1 Step 1: a list of scores	106
22.2 Step 2: walking the list	106
22.3 Step 3: names and scores in a map	107
22.4 Step 4: walking the map	107
22.5 Step 5: mixed data	108
22.6 What you practiced	108
23 Guide: A Multi-File Project with JBS	109
23.1 The layout	109
23.2 Step 1: the reusable code	109
23.3 Step 2: the program	110
23.4 Step 3: the project file	110
23.5 Step 4: build	110
23.6 Step 5: look inside	111
23.7 Step 6: configure the build	111
23.8 What you practiced	111
24 Guide: Calling C from Jaguar	112
24.1 The C side	112
24.2 Way 1: declare the function by hand	112
24.3 Way 2: generate the header with BindGen	113
24.4 Things to remember about C types	114
24.5 What you practiced	115
25 Recipes	116
25.1 Walk over the characters of a string	116
25.2 Count the digits in a string	116
25.3 Keep a value inside a range	117
25.4 Roll a die	117
25.5 Time a piece of code	117
25.6 Convert between text and numbers	117
25.7 Compare strings by content	117
25.8 Color the console output	118

25.9 Read or create a save file	118
25.10 Read an environment variable	118
25.11 Stop early with an error code	119
25.12 Check an assumption while developing	119
25.13 Enable code only in debug builds	119
25.14 Give a constructor a default	119
25.15 Run another program	120
26 Coming from C and C++	121
26.1 What stays the same	121
26.2 What is different	121
26.3 What is missing on purpose	122
26.4 A quick translation	122
27 Pitfalls and Troubleshooting	124
27.1 Loops	124
27.2 Collections	125
27.3 Types and casts	125
27.4 Functions	126
27.5 Classes	126
27.6 Scope and globals	127
27.7 Enums and interop	127
27.8 Build and tooling	128

Part VI • History and Roadmap

129

28 The Story of Jaguar	130
28.1 The starting idea: a language that compiles to C	130
28.2 A compiler in four stages	130
28.3 The signature decisions	131
28.4 How C interoperability grew up	131
28.5 From one compiler to a toolchain	132
28.6 The principles behind the choices	132
29 A Development Plan	134
29.1 Where Jaguar stands today	134
29.2 Boundaries to protect	135

29.3 Candidate work items	135
29.4 Proposed phases	136
29.5 How to know a phase is done	137

Appendices 138

A Language Quick Reference	139
A.1 Types	139
A.2 Control flow	140
A.3 Classes	140
A.4 Namespaces	140
A.5 Attributes	140
A.6 C interop	141
A.7 Preprocessor	141
A.8 Collections	141
A.9 System	142
A.10 jcc standard library	142
A.11 Syntax summary	143
B Command-Line and JBS Reference	145
B.1 jcc.py, the compiler	145
B.2 jbs.py, the build system	145
B.3 jbg.py, the binding generator	146
B.4 JBS quick reference	147
C Complete Example Programs	148
C.1 A program that uses many features	148
C.2 A small game program	149
D Implementation Checklist	151
D.1 How to read the status of a feature	151
D.2 Current syntax checklist	151
E Glossary	155

Index 157

Preface

Jaguar is a compiled programming language with a familiar C-like face and a deliberately small, explicit core. This book is the whole story: how to use it, how it works underneath, where its design came from, and where it can go next.

Who this book is for

You do not need to know C to read this book, but if you do, you will feel at home quickly: Jaguar keeps the braces, the semicolons and the explicit types, and it removes a good part of the sharp edges. The book is written for three kinds of readers:

- **newcomers** who want to learn the language from a first program to a multi-file project;
- **C and C++ programmers** who want to know exactly what is the same, what is different, and why;
- **contributors and tool authors** who need a precise picture of the compiler, the build system and the binding generator.

How this book is organized

The book is divided into six parts, followed by appendices and an index.

- **Part I, Getting Started**, introduces the language, walks you through compiling a first program and offers a guided tour of every major feature.
- **Part II, The Language** is the reference for the language itself: types, functions, control flow, classes, collections, namespaces and reflection.
- **Part III, The Standard Library** documents the two built-in libraries, `sys` and `jcc`.
- **Part IV, Interoperability and Tooling** covers calling C, generating bindings with BindGen, building projects with JBS and how the compiler works internally.
- **Part V, Guides** contains step-by-step tutorials, short recipes, a C/C++ comparison and a troubleshooting chapter.
- **Part VI, History and Roadmap** tells the design story of the language and proposes a development plan.

You can read the book front to back, but each reference chapter is written to stand on its own. If you are in a hurry, read the tour in Chapter 3 and then jump to the guides.

Conventions used in this book

Code appears in monospaced type. Short fragments such as `sys:print` or `for_loop` appear inline. Complete examples are set in blocks, and each block carries a small label in its upper-right corner:

```
// Jaguar source code
void main(string param) {
    sys:print("Hello");
}
```

JAGUAR

Commands that you type in a terminal are shown on a dark background:

```
python jcc.py main.ja -o game
```

SHELL

Project files and other text such as file listings and diagrams appear on the same light background, labelled accordingly. Occasionally a highlighted box draws attention to something particular:

NOTE

Notes add context or point to related material.

TIP

Tips suggest a better way of doing something.

WARNING

Warnings flag code that will not compile or behavior that is easy to get wrong.

DESIGN NOTE

Design notes explain *why* the language works the way it does.

COMING FROM C / C++

These boxes compare Jaguar with C and C++ for readers who know those languages.

About the documentation this book is built on

A REFERENCE TO THE CURRENT IMPLEMENTATION

The reference chapters of this book describe the compiler's **current implementation**, not a future specification. Some features may be intentionally limited. For any feature not described here, refer to the actual behavior of `gcc.py` rather than assuming that it is inherited from C or C++.

PART I

Getting Started

A first look at the language: what it is, how to compile a program, and a guided tour of its main ideas.

IN THIS PART

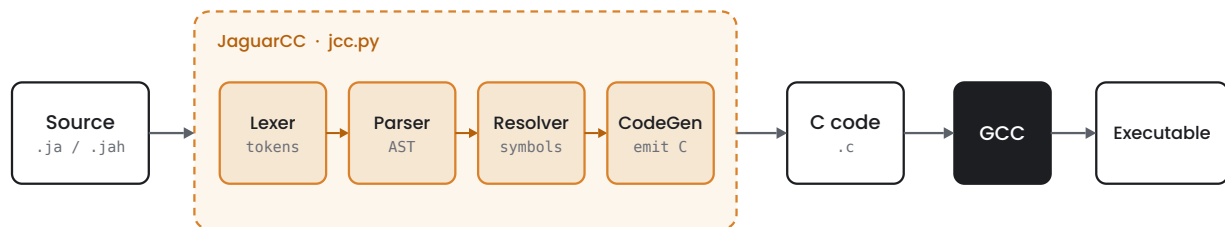
- 1 Meet Jaguar
- 2 Compiling Your First Program
- 3 A Tour of Jaguar

Meet Jaguar

Jaguar is a compiled language that reads like C, but with explicit types, classes, namespaces, simple built-in collections and a small standard library that keeps you away from raw pointers unless you ask for them.

1.1 What is Jaguar?

Jaguar is a compiled language that is translated to C by **JaguarCC** (`jcc.py`), then compiled with GCC. In other words, Jaguar is a source-to-source language: your `.ja` files become plain C, and the C compiler turns that into a native executable.



Every stage has a clear job. The lexer breaks the text into tokens, the parser builds a tree of the program, the resolver checks names, types, overloads and access rules, and the code generator writes the C. GCC does the rest. We look at each stage in detail in Chapter 19.

Jaguar deliberately borrows several ideas from C/C++ while simplifying certain aspects:

- syntax close to C;
- explicit types;
- classes and inheritance;
- function overloading;
- namespaces;
- simple collections;
- automatic management of certain objects;
- limited reflection;

- `jcc` standard library;
- C generation compatible with a C89 mode;
- interoperability with C APIs;
- Jaguar Headers (`.jah`);
- C binding generation with BindGen;
- JBS build system for multi-file projects and libraries.

1.2 Design philosophy

Jaguar tries to keep a balance between:

- **ease of use;**
- **syntax close to C/C++;**
- **performance of a compiled language;**
- **explicit type control;**
- **practical modern features;**
- **minimal runtime;**
- **interoperability with C.**

The compiler handles low-level details as much as possible while leaving the language's essential features explicit.

Those goals pull in different directions, and much of Jaguar's character comes from how it settles the tension. The result is a language where the common thing is short and the dangerous thing is explicit.

EXPLICIT BY DESIGN

Jaguar has explicit types, explicit casts (in C syntax only), explicit pointer syntax and an explicit `nullptr`. When something unusual happens, you can see it in the source.

1.3 What Jaguar deliberately leaves out

The general Jaguar language does not directly expose several low-level C/C++ mechanisms:

- general pointer arithmetic;
- `malloc/free` as a general Jaguar API;
- `FILE*`;
- `memcpy` and other memory primitives as a general Jaguar API;
- C++ templates;

- complex C++ features;
- lambdas;
- exceptions;
- multiple inheritance.

However, C interoperability provides a controlled set of low-level operations and types:

```
T*
fn(T1, T2, ...) -> R
&
*
nullptr
```

TEXT

They exist to allow using real C libraries without turning Jaguar into a general pointer-oriented language.

The details are documented in the C interoperability section.

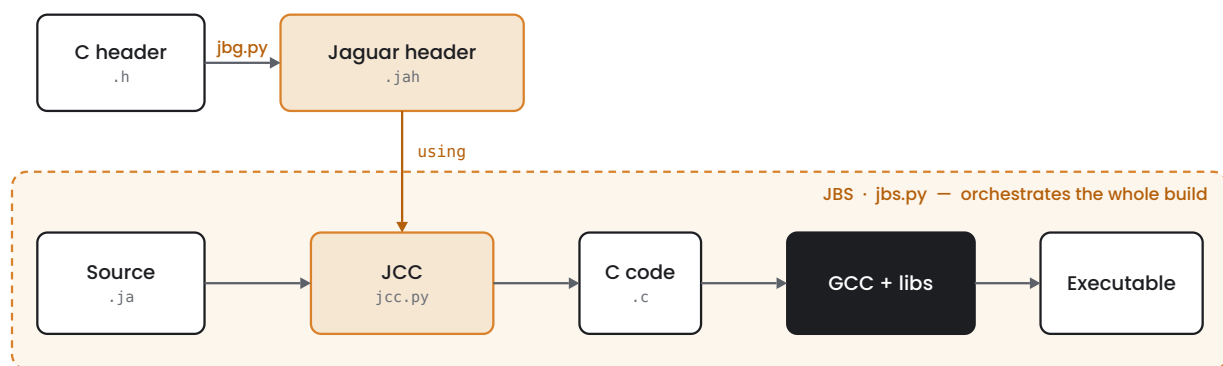
Features requiring more complex C mechanisms can be implemented in the C runtime or exposed via BindGen.

This list is not a to-do list. It is a set of boundaries, and it is what lets the rest of the language stay small.

1.4 The toolchain at a glance

Three command-line tools make up the Jaguar toolchain, and each one has a single responsibility:

TOOL	ROLE	INPUT	OUTPUT
<code>jcc.py</code>	The compiler (JaguarCC)	<code>.ja</code> , <code>.jah</code>	C code or an executable
<code>jbs.py</code>	The build system (JBS)	a <code>.jbs</code> project file	executables and libraries
<code>jbg.py</code>	The binding generator (BindGen)	a C header <code>.h</code>	a Jaguar header <code>.jah</code>



The compiler works on one compilation unit at a time. The build system gathers files, expands imports, calls the compiler and links the result. The binding generator reads C headers so that Jaguar programs can use existing C libraries safely. You will meet each of them in Part IV.

Compiling Your First Program

In this chapter you will write a two-line program, compile it, run it and understand what the compiler did on your behalf.

2.1 What you need

The Jaguar tools are Python scripts, so the first requirement is a working **Python 3** installation. Beyond that, each tool has its own needs:

- `jcc.py` translates Jaguar to C and, when you give it an output name, calls **GCC** automatically to produce the executable.
- `jbs.py` looks for a bundled MinGW toolchain (`gcc` and `ar`) relative to its own location, so it does not rely on a separately configured GCC in your `PATH`.
- `jbg.py` parses C headers with the `pycparser` Python package.

NOTE

This book assumes that the tools (`jcc.py`, `jbs.py`, `jbg.py`) are already available on your machine. The examples run them from a terminal with the `python` command.

2.2 Hello, Jaguar

Create a file named `hello.ja`:

```
void main(string param) {  
    sys:print("Hello, Jaguar!");  
}
```

JAGUAR

Compile it:

```
python jcc.py hello.ja -o hello
```

SHELL

and run the result. It prints:

```
Hello, Jaguar!
```

TEXT

That is the whole workflow. The compiler wrote `hello.c` next to your source, called GCC to build an executable named `hello`, and left both files in place.

Anatomy of the program

Even this tiny program shows several of Jaguar's habits.

- `void main(string param)` is the program's entry point. Jaguar translates this function into a standard C `main`; it never emits a non-standard C `main(string)`. A form returning an integer is also accepted.
- `sys:print(...)` calls a function from the built-in `sys` library. The colon is Jaguar's namespace separator, where C++ would write `::`.
- **Braces and semicolons.** Blocks use `{}` and statements generally end with `;`. Jaguar is not sensitive to whitespace or newlines.

2.3 Three ways to run the compiler

Compile straight to an executable

```
python jcc.py main.ja -o game
```

SHELL

The compiler generates:

```
game.c
game
```

TEXT

Then calls GCC automatically.

The output name **must not** end with `.c`:

```
python jcc.py main.ja -o game
```

SHELL

and not:

```
python jcc.py main.ja -o game.c
```

SHELL

Generate C only

Without `-o`, the C is written to standard output:

```
python jcc.py main.ja > main.c
```

SHELL

C89 mode

```
python jcc.py main.ja --c89 -o game
```

SHELL

In normal mode, declarations may appear after statements, as in C99+.

With `--c89`, Jaguar moves declarations to the beginning of blocks and leaves initialization at its logical location.

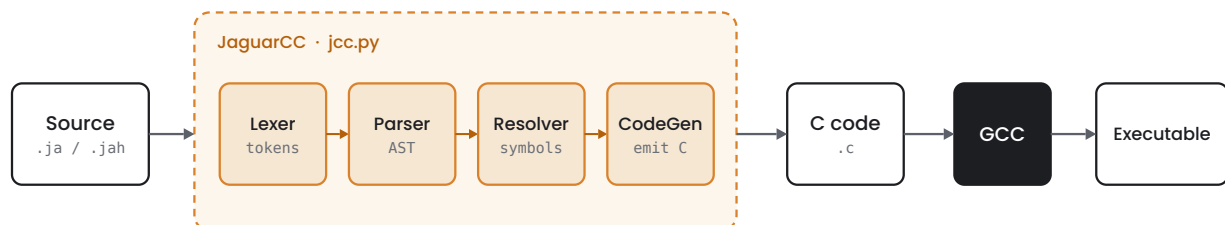
Jaguar semantics remain identical.

TIP

Get into the habit of reading the C that Jaguar produces. Run `python jcc.py hello.ja > hello.c` once and open the file: it is the most direct way to understand what a construct means.

2.4 What just happened?

When you ran `jcc.py`, your source went through four stages before GCC ever saw it.



The **lexer** turns text into tokens (comments are removed here). The **parser** organizes the tokens into a tree and records line numbers for diagnostics. The **resolver** looks up names, chooses function overloads, applies the numeric conversions the language allows and checks access rules. The **code generator** writes

C: methods receive an implicit `self`, `string` and the collections are handled by a small runtime, and names are mangled when namespaces or overloads require it.

If any stage finds a problem, you get an error pointing at the relevant line, and no executable is produced.

2.5 Command-line summary

COMMAND	WHAT IT DOES
<code>python jcc.py main.ja -o game</code>	Compiles to an executable named <code>game</code> and keeps <code>game.c</code>
<code>python jcc.py main.ja > main.c</code>	Writes the generated C to standard output
<code>python jcc.py main.ja --c89 -o game</code>	Same, with declarations moved to the start of blocks
<code>python jbs.py project.jbs</code>	Builds every target described in a JBS project
<code>python jbg.py --c api.h</code>	Generates <code>api.jah</code> from a C header

2.6 Where to go next

If you would like a fast overview of the whole language, continue with the tour in Chapter 3. If you prefer to go step by step, skip to Part II and start with Chapter 4.

A Tour of Jaguar

Here is the whole language in a dozen short stops. Every snippet in this tour is taken from, or built directly on, the reference material in Part II, so you can jump to the matching chapter whenever a stop makes you curious.

3.1 Variables and types

Jaguar has explicit, C-like types. Each integer and floating-point type has a short alias (`i32`, `u8`, `f64`, and so on), and `auto` lets the compiler deduce a local variable's type from its initializer.

```
int health = 100;
f64 speed = 12.5;
bool alive = true;
string name = "Oscar";

const int max_health = 100; // cannot be modified
auto ratio = 0.75;         // deduced as a decimal
```

JAGUAR

More in Chapter 5 and Chapter 6.

3.2 Functions

Parameters can have default values, arguments can be passed by name, and several functions may share a name if their parameters differ.

```
int add(int a, int b = 10) {
    return a + b;
}

void show(i32 value) {
    sys:print("integer:");
    sys:print(value);
}

void show(f64 value) {
    sys:print("decimal:");
}
```

JAGUAR

```

    sys:print(value);
}

void main(string param) {
    sys:print(add(5));           // 15
    sys:print(add(5, 20));       // 25
    sys:print(add(b = 1, a = 2)); // named arguments: 3
    show(10);                    // chooses show(i32)
    show(3.14);                  // chooses show(f64)
}

```

More in Chapter 7.

3.3 Control flow

Conditions use the familiar `if` / `else if` / `else`, and braces are mandatory. Loops are where Jaguar differs most from C: `while` takes no condition, and `for_loop` runs from a first value to a last value, **including** the last.

```

void main(string param) {
    int health = 60;

    if (health > 75) {
        sys:print("high");
    } else if (health > 25) {
        sys:print("medium");
    } else {
        sys:print("low");
    }

    int i = for_loop(0, 3) {
        sys:print(i);           // prints 0, 1, 2 and 3
    }

    while {
        health = health - 20;
        if (health <= 0) {
            break;
        }
    }
}

```

JAGUAR

More in Chapter 8.

3.4 Collections

`list<T>`, `map<K,V>`, `pair<K,V>` and `dynamic_list` are built into the language. Reading with `[]` is allowed; changes go through methods such as `push` and `emplace`. Two dedicated loops, `loop_list` and `loop_map`, walk a collection.

```
void main(string param) {
    list<int> numbers = {1, 2, 3, 4};
    numbers.push(5);

    map<string, int> scores = {
        "alice", 10,
        "bob", 20
    };
    scores.emplace("charlie", 30);

    auto n = loop_list(numbers) {
        sys:print(n);
    }

    auto entry = loop_map(scores) {
        sys:print(entry.first);
        sys:print(entry.second);
    }
}
```

JAGUAR

More in Chapter 11.

3.5 Classes

Classes have fields, methods, constructors (`constr`) and destructors (`destr`). Access control uses one-character markers: `$` for public and `%` for protected, with private as the default.

```
class Player {
    $int health = 100;

    constr(int initial_health) {
        health = initial_health;
    }

    void damage(int amount) {
        health = health - amount;
    }

    bool alive() const {
        return health > 0;
    }
}
```

JAGUAR

```
}

void main(string param) {
    Player player = Player(100);
    player.damage(25);
    if (player.alive()) {
        sys:print("Player alive");
    }
}
```

3.6 Inheritance and virtual methods

A derived class names its base class after a comma. A **virtual** method can be replaced by an **override**.

```
class Enemy {
    virtual void attack() {
        sys:print("enemy attack");
    }
}

class Zombie, Enemy {
    void attack() override {
        sys:print("zombie attack");
    }
}
```

JAGUAR

More on both in Chapter 10.

3.7 Namespaces

Namespaces group functions and types. The separator is a colon, and namespaces can be nested.

```
namespace math {
    int add(int a, int b) {
        return a + b;
    }
}

void main(string param) {
    int value = math:add(10, 20);
    sys:print(value);
}
```

JAGUAR

More in Chapter 12.

3.8 The standard library

Two libraries are built in. `sys` talks to the system (printing, console colors, files, running programs) and `jcc` is a simplified, libc-inspired library for math, strings, characters and the environment.

```
void main(string param) {
    f64 distance = jcc:sqrt(3.0 * 3.0 + 4.0 * 4.0);
    sys:print(distance);

    string text = jcc:i32_to_string(42);
    sys:print(jcc:string_concat("answer: ", text));
}
```

JAGUAR

More in Chapter 14 and Chapter 15.

3.9 Talking to C

Jaguar can call C functions with `@extern`, which keeps the exact C name. For larger APIs, BindGen turns a C header into a Jaguar header that you import with `using`.

```
@extern
int add(int a, int b);
```

JAGUAR

More in Chapter 16 and Chapter 17.

3.10 Building projects

Once a program outgrows one file, describe it in a `.jbs` file and let JBS do the work.

```
version 1.0

out build

compile Main {
    main.ja
}
```

JBS

```
python jbs.py project.jbs
```

SHELL

More in Chapter 18.

3.11 Reflection

A class marked `@register` can be created from its name at runtime, and its `@exposed` fields can be read and written dynamically.

```
@register
class Enemy {
    @exposed
    $int health = 100;
}

void main(string param) {
    auto enemy = factory:construct("Enemy");
    enemy.SetMember("health", 25);
    sys:print((int)enemy.GetMember("health"));
}
```

JAGUAR

More in Chapter 13.

KEEP GOING

The best way to learn Jaguar is to type these examples in, compile them, and change one thing at a time. The guides in Part V walk you through five complete programs.

PART II

The Language

The reference for Jaguar itself: types, functions, control flow, classes, collections, namespaces and reflection.

IN THIS PART

- 4** Program Structure and Lexical Elements
- 5** Types, Variables and Operators
- 6** Strings
- 7** Functions
- 8** Control Flow and Scope
- 9** Structs, Enums, Unions and Aliases
- 10** Classes and Objects
- 11** Collections and Object Ownership
- 12** Namespaces and Modules
- 13** Reflection, Attributes and Signals

Program Structure and Lexical Elements

Before types and functions, a few ground rules about what a Jaguar source file looks like and how the compiler reads it.

4.1 General syntax

Jaguar is **not** whitespace/newline sensitive.

Blocks use `{}` and statements generally end with `;`.

Example:

```
int main(string param) {  
    int x = 10;  
    int y = 20;  
  
    if (x < y) {  
        sys:print("x is smaller");  
    }  
  
    return 0;  
}
```

JAGUAR

4.2 Source files

Jaguar mainly uses two extensions:

- `.ja`: Jaguar source file;
- `.jah`: **Jaguar Header**, a Jaguar interface file.

Technically, a `.jah` uses the same syntax as Jaguar. The extension simply indicates that it is an interface meant to be reused with `using`.

Example:

```
include/  
    math.jah  
src/  
    main.ja
```

TEXT

Then:

```
using math;  
  
void main(string param) {  
    math:add(10, 20);  
}
```

JAGUAR

`.jah` files are especially used for C API bindings generated by BindGen.

4.3 Comments

Single-line comment

```
// This is a comment  
int x = 10;
```

JAGUAR

Multi-line comment

```
/*  
    Comment  
    spanning multiple lines  
*/
```

JAGUAR

Documentation comments `/** ... */` are also stripped by the lexer.

4.4 Literals

Integers

```
0
42
-10
```

JAGUAR

Decimals

```
3.14
0.5
10.0
```

JAGUAR

Booleans

```
true
false
```

JAGUAR

Strings

```
"Hello world"
```

JAGUAR

4.5 The preprocessor

Jaguar keeps certain C preprocessor directives and passes them through to the generated C. `#include` is **not supported**: C headers must be brought in with BindGen/Jaguar headers.

The supported directives are:

```
#define
#undef

#if
#ifdef
#ifndef
#elif
#else
#endif
```

TEXT

```
#pragma
#error
#warning
#line
```

Example:

```
#define PI 3.14159265
#define GAME_VERSION 1

#ifdef JAGUAR_DEBUG
    // debug code
#elif defined(JAGUAR_RELEASE)
    // release code
#else
    // other configuration
#endif
```

JAGUAR

Jaguar also accepts `#elseif` as an alternative spelling of `#elif`; it is normalized to `#elif` in the generated C.

Conditions are evaluated by the C preprocessor during GCC compilation. Blocks can be nested.

`#include` is deliberately forbidden:

```
#include <stdio.h> // invalid
```

JAGUAR

To use a C API, use a `.jah` file generated by BindGen along with `using`.

Macros are not Jaguar variables and their type is not inferred by the compiler.

```
#define START_HEALTH 100

int default_health = START_HEALTH;
```

JAGUAR

4.6 Top-level expressions

The current parser accepts a top-level expression statement, notably a call:

```
initialize();
```

JAGUAR

or:

```
engine:start();
```

JAGUAR

JCC emits the corresponding call at global scope when the expression is valid in the generated C context.

This is separate from normal function bodies.

Types, Variables and Operators

Jaguar is statically typed, with a small set of built-in types that map directly onto C types, plus a few types that are managed by the runtime.

5.1 Core types

Every numeric type has a short alias that spells out its size. The compiler normalizes aliases, so `int` and `i32` are the same logical type.

JAGUAR	ALIAS	APPROXIMATE C TYPE
<code>void</code>	<code>none</code>	<code>void</code>
<code>int</code>	<code>i32</code>	<code>int</code>
<code>uint</code>	<code>u32</code>	<code>unsigned int</code>
<code>short</code>	<code>i16</code>	<code>short</code>
<code>ushort</code>	<code>u16</code>	<code>unsigned short</code>
<code>long</code>	<code>i64</code>	<code>long long</code>
<code>ulong</code>	<code>u64</code>	<code>unsigned long long</code>
<code>char</code>	<code>i8</code>	<code>signed char</code>
<code>uchar</code>	<code>u8</code>	<code>unsigned char</code>
<code>sbyte</code>	<code>i8</code>	<code>signed char</code>
<code>byte</code>	<code>u8</code>	<code>unsigned char</code>
<code>float</code>	<code>f32</code>	<code>float</code>
<code>double</code>	<code>f64</code>	<code>double</code>
<code>bool</code>	<code>none</code>	<code>_jBool</code>
<code>string</code>	<code>none</code>	Jaguar string object

JAGUAR

```
int a = 10;
i32 b = 20;    // exactly the same logical type as int
```

SIZED NAMES FOR PORTABLE CODE

Prefer the sized aliases (`i32`, `u64`, `f64`) when the width matters, for example when talking to C code. Use the C-like names (`int`, `long`, `double`) when it does not. Both spellings are always equivalent.

5.2 Variables

Declaration

JAGUAR

```
int x;
f64 speed;
bool alive;
string name;
```

Initialization

JAGUAR

```
int x = 42;
f64 speed = 12.5;
bool alive = true;
string name = "Oscar";
```

A local variable can be declared with any known type.

5.3 Constants with `const`

`const` makes a variable or parameter non-modifiable.

JAGUAR

```
const int max_health = 100;
```

A parameter can also be `const`:

JAGUAR

```
void print_value(const int value) {
    sys:print(value);
}
```

Methods can also be declared `const`:

```
int get_health() const {
    return health;
}
```

JAGUAR

A local `const` variable must be initialized immediately.

```
const int x = 10;
```

JAGUAR

5.4 Type inference with `auto`

`auto` deduces the type of a local variable from its initializer.

```
auto x = 42;
auto value = 3.14;
auto name = "hello";
```

JAGUAR

An `auto` variable must be initialized.

```
auto x; // invalid
```

JAGUAR

`auto` is intended for local variables and the loop constructs provided by Jaguar.

5.5 Global variables

A variable can be declared at global scope:

```
int max_health = 100;
f64 gravity = 9.81;
```

JAGUAR

A global initializer must be a constant expression:

- literal;
- `#define` macro;
- operation between constants.

This is valid:

```
#define MAX_HEALTH 100
int max_health = MAX_HEALTH;
```

JAGUAR

This is not valid as a global initialization:

```
int x = get_value();
```

JAGUAR

Global variables must also be declared before use.

`list`, `map`, and `container` are currently not usable as global variables.

5.6 Operators

Arithmetic

```
+ - * / %
```

TEXT

Example:

```
int result = (10 + 5) * 2;
```

JAGUAR

Comparisons

```
== != < > <= >=
```

TEXT

Comparisons produce a `bool`.

```
bool result = health <= 0;
```

JAGUAR

Logical

```
&& || !
```

TEXT

Example:

```
if (alive && health > 0) {
    sys:print("alive");
}
```

JAGUAR

Precedence

Precedence is close to C's:

```
||
&&
== !=
< > <= >=
+ -
* / %
- !
```

TEXT

Parentheses can be used to make an expression explicit.

5.7 Casts

Jaguar deliberately uses C cast syntax:

```
f64 value = (f64)10;
i32 integer = (i32)3.14;
```

JAGUAR

The following form is **not** a Jaguar cast:

```
i32(3.14); // this is not cast syntax
```

JAGUAR

5.8 Implicit numeric conversion in calls

The resolver can use simple numeric conversions to choose an overload:

- integer → other integer type;
- integer → float;
- float → other float type.

Example:

```
void test(i32 value) {  
    sys:print(value);  
}  
  
void test(f64 value) {  
    sys:print(value);  
}  
  
test(10);  
test(3.14);
```

JAGUAR

If several overloads remain equally valid, the call is considered ambiguous.

Strings

`string` is a built-in type with its own methods. It is managed by the Jaguar runtime, which means you never allocate or free a string by hand.

6.1 Methods at a glance

METHOD	RETURNS	PURPOSE
<code>length()</code>	<code>int</code>	Number of characters
<code>empty()</code>	<code>bool</code>	True when the string has no characters
<code>equals(other)</code>	<code>bool</code>	Compares the contents of two strings
<code>contains(text)</code>	<code>bool</code>	Tests whether <code>text</code> appears inside the string
<code>starts_with(text)</code>	<code>bool</code>	Tests the beginning of the string
<code>ends_with(text)</code>	<code>bool</code>	Tests the end of the string
<code>concat(other)</code>	<code>string</code>	Returns the two strings joined
<code>substring(...)</code>	<code>string</code>	Extracts part of the string
<code>char_at(index)</code>	<code>i32</code>	Character code at an index, or <code>-1</code> if the index is invalid
<code>to_upper()</code> / <code>to_lower()</code>	<code>string</code>	Returns a case-converted copy

Every method is described with an example below.

6.2 The `string` type

`string` is Jaguar's native string type.

```
string name = "Jaguar";
```

JAGUAR

It is not a plain `char*` exposed to the Jaguar program.

Available methods

length()

```
int n = name.length();
```

JAGUAR

empty()

```
bool empty = name.empty();
```

JAGUAR

equals()

```
if (name.equals("Jaguar")) {  
    sys:print("OK");  
}
```

JAGUAR

contains()

```
bool found = name.contains("gua");
```

JAGUAR

starts_with()

```
bool result = name.starts_with("Jag");
```

JAGUAR

ends_with()

```
bool result = name.ends_with("ar");
```

JAGUAR

concat()

```
string full = name.concat("CC");
```

JAGUAR

substring()

```
string part = name.substring(0, 3);
```

JAGUAR

char_at()

```
i32 c = name.char_at(0);
```

JAGUAR

The result is the character code, or **-1** if the index is invalid.

to_upper() / to_lower()

```
string upper = name.to_upper();
string lower = name.to_lower();
```

JAGUAR

6.3 Constructing an empty string

The identifier **string** has a special zero-argument construction form:

```
string text = string();
```

JAGUAR

The current implementation maps this to the Jaguar runtime string constructor.

A non-empty argument list is rejected; use a string literal or the documented string operations instead.

COMING FROM C / C++

A Jaguar **string** is not a **char***. When you need a C string for a native API, use the **i8*** type described in Chapter 16.

Functions

Functions are the workhorse of a Jaguar program. They look like C functions, and they add default parameters, named arguments and overloading.

7.1 Declaring and calling functions

Declaration

```
int add(int a, int b) {  
    return a + b;  
}
```

JAGUAR

Call

```
int result = add(10, 20);
```

JAGUAR

void

```
void say_hello() {  
    sys:print("Hello");  
}
```

JAGUAR

7.2 Default parameters

A parameter can have a default value:

```
int add(int a, int b = 10) {  
    return a + b;  
}
```

JAGUAR

Both of the following calls are valid:

```
add(5);
add(5, 20);
```

JAGUAR

7.3 Named arguments

Arguments can be passed by name:

```
int move(int x, int y, int speed = 1) {
    return x + y + speed;
}

move(y = 20, x = 10);
move(x = 10, y = 20, speed = 5);
```

JAGUAR

Once a named argument is used, a positional argument can no longer follow.

```
move(x = 10, 20); // invalid
```

JAGUAR

The built-in `jcc.*` and `sys.*` functions do not accept named arguments.

7.4 Prototypes

A function can be declared without a body:

```
int add(int a, int b);
```

JAGUAR

Then defined later:

```
int add(int a, int b) {
    return a + b;
}
```

JAGUAR

The prototype and the definition must have exactly the same signature.

Two identical prototypes or two identical definitions cause an error.

7.5 Overloading

Jaguar allows several functions with the same name and different parameters:

```
void print_value(i32 value) {  
    sys:print(value);  
}  
  
void print_value(f64 value) {  
    sys:print(value);  
}
```

JAGUAR

The compiler chooses the appropriate overload.

Name mangling uses the parameter types when the name is actually overloaded.

7.6 How a call is resolved

When a call is encountered, Jaguar looks up:

1. the namespace and the name;
2. the candidate functions;
3. default parameters;
4. named arguments;
5. the argument types;
6. the allowed numeric conversions;
7. the best-matching overload.

An ambiguous call causes an error.

7.7 The `main` function

The recommended special form is:

```
void main(string param) {  
    sys:print("Hello");  
}
```

JAGUAR

or a form returning an integer, depending on the functions supported by the program.

Jaguar translates this function into a standard C `main`.

The compiler avoids emitting a non-standard C `main(string)`.

7.8 Returning values

```
int square(int x) {  
    return x * x;  
}
```

JAGUAR

For a `void` function:

```
void stop() {  
    return;  
}
```

JAGUAR

7.9 Native functions with `@extern`

`@extern` prevents name mangling.

```
@extern  
int my_native_function(int value);
```

JAGUAR

The C name stays exactly `my_native_function`.

This notably allows calling functions defined elsewhere in the project or in a native library.

TIP

Named arguments and default parameters work together nicely: give a function sensible defaults, then override only the arguments you care about, by name.

Control Flow and Scope

Conditions look like C. Loops are where Jaguar goes its own way: `while` has no condition, and `for_loop` includes its last value.

8.1 Conditions

`if`

```
if (health > 0) {  
    sys:print("alive");  
}
```

JAGUAR

`else`

```
if (health > 0) {  
    sys:print("alive");  
} else {  
    sys:print("dead");  
}
```

JAGUAR

`else if`

```
if (health > 75) {  
    sys:print("high");  
} else if (health > 25) {  
    sys:print("medium");  
} else {  
    sys:print("low");  
}
```

JAGUAR

Braces are mandatory.

8.2 The `while` loop

Jaguar's `while` is deliberately different from C's: it takes **no condition**.

```
while {
    if (health <= 0) {
        break;
    }

    health = health - 1;
}
```

JAGUAR

Exiting the loop is done with `break` or `return`.

The following form is forbidden:

```
while (health > 0) {
}
```

JAGUAR

THE CONDITION MOVES INSIDE THE LOOP

In C, `while (health > 0) { ... }` tests the condition before each iteration. In Jaguar, `while` takes no condition. You write the exit test yourself, and leave with `break`:

```
while {
    health = health - 10;
    if (health <= 0) {
        break;
    }
}
```

JAGUAR

8.3 `break` and `continue`

```
while {
    if (value == 10) {
        break;
    }

    if (value == 5) {
        continue;
    }
}
```

JAGUAR

They are only valid inside a loop.

8.4 The `for_loop` loop

Jaguar provides a simplified numeric loop:

```
int i = for_loop(0, 10) {
    sys:print(i);
}
```

JAGUAR

The final bound is **included**.

So:

```
0 1 2 3 4 5 6 7 8 9 10
```

TEXT

If the start is greater than the end, no iteration is performed.

The index type must be an integer:

```
int / i8 / u8 / i16 / u16 / i32 / u32 / i64 / u64
```

TEXT

The index is read-only inside the loop.

THE LAST VALUE IS INCLUDED

`for_loop(0, 10)` runs eleven times: 0, 1, ..., 10. When you walk over a string or a list by index, remember to stop at `length - 1`.

8.5 Variable scope

Each block has its own scope.

```
void test() {
    int x = 10;

    if (true) {
        int y = 20;
        sys:print(y);
    }

    // y no longer exists here
}
```

JAGUAR

Two sibling blocks can use the same name:

```
if (true) {  
    int x = 10;  
}  
  
if (true) {  
    int x = 20;  
}
```

JAGUAR

On the other hand, shadowing a variable visible from an enclosing scope is forbidden.

Structs, Enums, Unions and Aliases

Beyond classes, Jaguar has the classic C data-definition tools: plain structs, enumerations, unions and type aliases. They matter most when you work with C APIs.

9.1 Structs

Jaguar structures are simple data structures.

```
struct Vector2 {  
    f32 x;  
    f32 y;  
}
```

JAGUAR

Usage:

```
Vector2 position;  
position.x = 10.0;  
position.y = 20.0;
```

JAGUAR

They are generated as C `typedef struct`.

9.2 Enumerations

Jaguar supports sequential enums:

```
enum Color {  
    Red,  
    Green,  
    Blue  
}
```

JAGUAR

Values are automatically numbered sequentially by the generated C enum.

Explicit enum initializers are not currently accepted:

```
enum Color {
    Red = 10 // invalid
}
```

JAGUAR

Enums can also be declared inside namespaces:

```
namespace graphics {
    enum Format {
        RGB,
        RGBA
    }
}
```

JAGUAR

The generated C enum and its values receive the namespace prefix needed to avoid collisions.

9.3 Unions

Jaguar supports named unions:

```
union Value {
    int integer;
    f32 real;
}
```

JAGUAR

A named union also provides its implicit global union storage, so this form is valid:

```
union Value {
    int integer;
    f32 real;
}

void main(string param) {
    Value.integer = 42;
    sys:print((int)Value);
}
```

JAGUAR

The union storage is emitted internally with a private C name, while Jaguar code accesses it through the union name.

A union cast is special: when casting a union value, the target type must match one of the union members.

```
sys:print((int)Value);  
sys:print((f32)Value);
```

JAGUAR

Casting the union to a type that is not one of its members is a compiler error.

9.4 Forward declarations

The current parser accepts forward declarations for classes, structs, and unions:

```
class Player;  
struct Node;  
union Value;
```

JAGUAR

These are useful when a pointer or prototype needs to refer to a type before its full definition.

9.5 Type aliases with `using`

A type alias can be declared with:

```
using Number = f64;
```

JAGUAR

Aliases can be used anywhere a normal type is accepted.

Aliases can also point to generic types:

```
using Scores = map<string, i32>;
```

JAGUAR

Circular aliases are rejected.

Function-pointer aliases use the same mechanism:

```
using Callback = fn(i32) -> void;
```

JAGUAR

Classes and Objects

Classes bundle data and behavior. Jaguar's class system is compact: fields, methods, constructors, destructors, single inheritance and virtual dispatch, with access control written as a single character.

10.1 Declaring a class

Jaguar has a class system with:

- fields;
- methods;
- constructors;
- destructors;
- inheritance;
- virtual methods;
- `override`;
- access control.

Example:

```
class Player {  
    $int health = 100;  
  
    void take_damage(int amount) {  
        health = health - amount;  
    }  
  
    int get_health() const {  
        return health;  
    }  
}
```

JAGUAR

10.2 Access control

Members are private by default.

Public: `$`

```
class Player {  
    $int health;  
}
```

JAGUAR

Protected: `%`

```
class Player {  
    %int health;  
}
```

JAGUAR

Private

Without a marker:

```
class Player {  
    int secret_value;  
}
```

JAGUAR

Access control is checked by the compiler.

10.3 Methods

```
class Player {  
    $int health;  
  
    void damage(int amount) {  
        health = health - amount;  
    }  
}
```

JAGUAR

Call:

```
Player player = Player();
player.damage(10);
```

JAGUAR

A method implicitly has a `self` in the generated C.

10.4 The `this` keyword

Inside a class, `this` refers to the current instance.

```
class Player {
    $int health;

    void reset() {
        this.health = 100;
    }
}
```

JAGUAR

In the generated C, `this` corresponds to `self`.

`this` is only available inside a class.

10.5 Constructors with `constr`

```
class Player {
    $int health;

    constr(int initial_health) {
        health = initial_health;
    }
}
```

JAGUAR

Creation:

```
Player player = Player(100);
```

JAGUAR

The compiler also generates the object allocation.

10.6 Destructors with `destr`

```
class Player {
    destr() {
        sys:print("destroyed");
    }
}
```

JAGUAR

Every class has a destructor entry point generated by the compiler, even if no `destr()` is written.

With inheritance, the base class's destructor is called automatically.

10.7 Inheritance

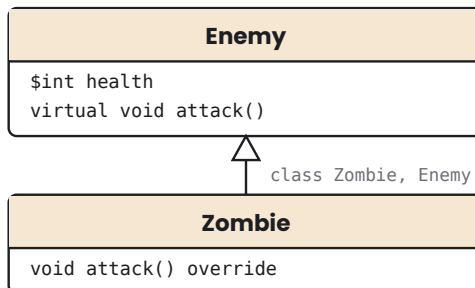
The current syntax is:

```
class Enemy {
    $int health;
}

class Zombie, Enemy {
    void attack() {
        health = health - 10;
    }
}
```

JAGUAR

The derived class contains the part corresponding to its base class.



10.8 Virtual methods

A method can be virtual:

```
class Enemy {
    virtual void attack() {
        sys:print("enemy attack");
    }
}
```

JAGUAR

A derived class can override it:

```
class Zombie, Enemy {
    void attack() override {
        sys:print("zombie attack");
    }
}
```

JAGUAR

`override` must match a `virtual` method of the base class with an exact signature.

10.9 `const` methods

```
class Player {
    $int health;

    int get_health() const {
        return health;
    }
}
```

JAGUAR

Parameters can also be `const`.

COMING FROM C / C++

Class syntax will look familiar, with a few substitutions: `constr` and `destr` replace the class-named constructor and the `~` destructor, the one-character markers `$` and `%` replace the `public:` and `protected:` labels, a base class follows a comma (`class Zombie, Enemy`), and `override` comes after the parameter list.

Collections and Object Ownership

Jaguar has its collections built in. You do not import a library or write template arguments by hand: `list`, `map`, `pair` and friends are part of the language, and the runtime manages their memory.

11.1 The collection types

TYPE	HOLDS	TYPICAL USE
<code>list<T></code>	An ordered sequence of <code>T</code>	Sequences of numbers, names, IDs
<code>map<K,V></code>	Key/value entries	Lookups, scoreboards, settings
<code>pair<K,V></code>	Two values, <code>first</code> and <code>second</code>	The entries produced by <code>loop_map</code>
<code>dynamic_list</code>	Values of several different types	Mixed data such as <code>{42, "hello", true}</code>
<code>container<T></code>	One value whose data the container owns	Objects created with <code>new</code> , destroyed automatically

NOTE

Collections are **created with literals, changed with methods, read with indexing and walked with dedicated loops**. Indexing is read-only, so `numbers[2] = 10;` is not valid; see the section on indexing below. In the current implementation, collections are kept as local variables.

11.2 Lists

`list<T>` is a homogeneous collection.

```
list<int> numbers = {1, 2, 3, 4};
```

JAGUAR

Adding an element:

```
numbers.push(5);
```

JAGUAR

Reading an element:

```
int value = numbers[2];
```

JAGUAR

Indexing is **read-only**.

The following syntax is not allowed:

```
numbers[2] = 10;
```

JAGUAR

To modify/add values, use the operations provided by the collection.

11.3 Maps

`map<K, V>` is a key/value collection.

```
map<string, int> scores = {
    "alice", 10,
    "bob", 20
};
```

JAGUAR

Adding or replacing an entry:

```
scores.emplace("charlie", 30);
```

JAGUAR

Reading a value:

```
int score = scores["alice"];
```

JAGUAR

Indexing is read-only:

```
scores["alice"] = 100; // invalid
```

JAGUAR

To write into the map, use `emplace()`.

The `string` key has content-based comparison.

11.4 Pairs

A pair contains two values: `first` and `second`.

```
pair<string, int> result = {"score", 42};

sys::print(result.first);
sys::print(result.second);
```

JAGUAR

`pair` is notably used by `loop_map`.

11.5 Dynamic lists

`dynamic_list` allows storing several types in the same collection.

```
dynamic_list values = {
    42,
    "hello",
    true,
    3.14
};
```

JAGUAR

Adding:

```
values.push(123);
values.push("world");
```

JAGUAR

Size:

```
int count = values.size();
```

JAGUAR

Type of a value:

```
string type = values.type(1);
```

JAGUAR

Raw access:

```
auto value = values.get(0);
```

JAGUAR

Indexing is also available:

```
sys:print(values[0]);
```

JAGUAR

To extract a primitive value with a known type, a cast can be used:

```
i32 number = (i32)values[0];
f64 decimal = (f64)values[3];
```

JAGUAR

For objects/classes, the cast keeps the pointer to the stored object.

11.6 Creating collections with {}

Braces can create collection literals.

List:

```
list<int> values = {1, 2, 3};
```

JAGUAR

Map:

```
map<string, int> values = {
    "one", 1,
    "two", 2
};
```

JAGUAR

Pair:

```
pair<string, int> value = {"score", 42};
```

JAGUAR

dynamic_list:

```
dynamic_list values = {1, "hello", true};
```

JAGUAR

11.7 Walking a list with `loop_list`

`loop_list` iterates over a `list<T>`.

```
list<int> numbers = {1, 2, 3, 4};

auto item = loop_list(numbers) {
    sys::print(item);
}
```

JAGUAR

During the loop, the collection is treated as read-only.

The following is therefore forbidden:

```
auto item = loop_list(numbers) {
    numbers.push(10); // forbidden
}
```

JAGUAR

`loop_list` does not work on `dynamic_list`.

11.8 Walking a map with `loop_map`

`loop_map` iterates over a `map<K,V>` and provides a `pair<K,V>`.

```
map<string, int> scores = {
    "alice", 10,
    "bob", 20
};

auto item = loop_map(scores) {
    sys::print(item.first);
    sys::print(item.second);
}
```

JAGUAR

The map is read-only during the loop.

11.9 Indexing

`[]` is available on certain collections.

```
int x = numbers[0];
int score = scores["player"];
auto value = values[0];
```

JAGUAR

It is deliberately read-only.

`container<T>` values are not indexable.

11.10 Creating objects with `new`

Jaguar has the `new` expression:

```
new Player(100)
```

JAGUAR

It can be used in `container` ownership mechanisms.

11.11 Containers

`container<T>` represents a value whose data is owned by the container.

```
container<int> value = new int(42);
```

JAGUAR

A short-hand syntax is available:

```
container<int> value => 42;
```

JAGUAR

Reading:

```
int x = value.get();
```

JAGUAR

For a class:

```
container<Player> player = new Player(100);
player.get().damage(10);
```

JAGUAR

The container automatically destroys the object it owns.

`container` values cannot be indexed with `[]`.

11.12 Memory management

Jaguar deliberately does not give direct access to the entire memory-related part of libc.

The runtime notably manages:

- strings;
- classes;
- containers;
- lists;
- maps;
- dynamic lists.

Internal structures use C `malloc`, `calloc`, `realloc`, `free`, etc., but these pointers are not exposed as a general Jaguar API.

This keeps Jaguar syntax safer and simpler.

11.13 Where collections can and cannot be used

The collection types are compiler/runtime features rather than C++ templates:

```
list<T>
map<K, V>
pair<K, V>
container<T>
dynamic_list
```

TEXT

The current implementation keeps these collections as local variables. In particular, generic collection types are not currently valid as global variables.

Namespaces and Modules

Namespaces keep names apart, and `using` brings code from other files and namespaces into scope. Together they let a Jaguar project grow past a single file.

12.1 Namespaces

Namespaces use `::`.

```
namespace math {  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

JAGUAR

Call:

```
int value = math::add(10, 20);
```

JAGUAR

Namespaces can be nested:

```
namespace engine {  
    namespace math {  
        int add(int a, int b) {  
            return a + b;  
        }  
    }  
}
```

JAGUAR

Call:

```
engine::math::add(10, 20);
```

JAGUAR

The generated C name is mangled to avoid collisions.

Namespaces currently support functions, structs, classes, and nested namespaces according to the compiler's rules. Global variables directly inside a namespace are not supported.

12.2 Importing a namespace

```
using namespace math;
```

JAGUAR

After this declaration:

```
int value = add(10, 20);
```

JAGUAR

can resolve to a function in `math` when the namespace lookup is unambiguous.

Nested namespaces can be imported:

```
using namespace engine::math;
```

JAGUAR

12.3 Importing one symbol, with an alias

A specific symbol can be imported:

```
using math::add;
```

JAGUAR

It can also be renamed:

```
using math::add as sum;
```

JAGUAR

Then:

```
int value = sum(10, 20);
```

JAGUAR

resolves to the imported function.

The same mechanism is used by the resolver for imported enum symbols.

12.4 Importing a complete Jaguar file

The form:

```
using math;
```

JAGUAR

requests a Jaguar file named `math.ja` or `math.jah` when processed by JBS.

The form is different from:

```
using namespace math;
```

JAGUAR

which imports namespace lookup into the current compilation unit.

NOTE

A `using` line can mean three different things: importing a file, opening a namespace, or importing a single symbol. Jaguar header files (`.jah`), which are the usual target of a file import, are covered in Chapter 17.

Reflection, Attributes and Signals

Reflection lets a program create objects by name and read or write their fields at runtime. It is opt-in: a class must declare itself with `@register`, and each field it exposes must say so with `@exposed`.

13.1 Overview

Jaguar has a deliberately limited reflection system.

A class can be registered with `@register`:

```
@register
class Player {
    $int health = 100;
}
```

JAGUAR

A public field can be exposed with `@exposed`:

```
@register
class Player {
    @exposed
    $int health = 100;
}
```

JAGUAR

`@exposed` can only be used on a public field.

13.2 Creating objects with `factory:construct`

A registered class can be constructed dynamically by name:

```
string class_name = "Player";
auto object = factory:construct(class_name);
```

JAGUAR

The class must be registered with `@register`.

The name is resolved at runtime.

If the class is not registered, the runtime produces an error.

13.3 Reading a member with `GetMember`

An exposed member can be retrieved dynamically:

```
string member = "health";
int health = (int)object.GetMember(member);
```

JAGUAR

The member name is a `string`.

If the member does not exist or is not exposed, the runtime raises an error.

The cast is important to get a typed value.

13.4 Testing a member with `MemberExists`

Before a dynamic access:

```
string name = "health";

if (object.MemberExists(name)) {
    int health = (int)object.GetMember(name);
}
```

JAGUAR

This allows checking whether a member is exposed.

13.5 Writing a member with `SetMember`

An exposed member can be modified dynamically:

```
object.SetMember("health", 50);
```

JAGUAR

Primitive values and `string` are supported by the reflection runtime.

13.6 A complete reflection example

```
@register
class Enemy {
    @exposed
    $int health = 100;

    $string name = "Enemy";
}

void main(string param) {
    string class_name = "Enemy";
    auto enemy = factory.construct(class_name);

    if (enemy.MemberExists("health")) {
        sys.print((int)enemy.GetMember("health"));
        enemy.SetMember("health", 25);
        sys.print((int)enemy.GetMember("health"));
    }
}
```

JAGUAR

Only members marked `@exposed` are accessible via reflection.

13.7 Attributes

The attributes currently recognized by the compiler are:

```
@extern
@register
@exposed
```

TEXT

Their use depends on context:

An unknown attribute produces a compilation error.

The three attributes and their targets are summarized here.

ATTRIBUTE	APPLIES TO
<code>@extern</code>	a function or a global variable
<code>@register</code>	a class
<code>@exposed</code>	a public class field

13.8 Signals: reacting to a variable change

Jaguar has a `signal:` syntax for attaching a block to a variable:

```
signal:health {
    sys:print("Health changed");
}
```

JAGUAR

This feature is meant to attach a reaction to changes in a variable.

NOTE

the exact behavior depends on the code generation currently implemented in `jcc.py`. This syntax is part of the compiler's AST and should be regarded as a Jaguar-specific feature, not a standard C construct.

PART III

The Standard Library

Two built-in libraries: sys for talking to the operating system, and jcc for math, strings, characters and the environment.

IN THIS PART

- 14** The sys Library
- 15** The jcc Library

The `sys` Library

`sys` is Jaguar's window on the operating system: printing to the console, coloring output, running other programs and reading or writing files. Everything is reached through the `sys:` prefix.

14.1 Printing

`sys` provides system functions built directly into JaguarCC.

`sys:print`

```
sys:print("Hello");  
sys:print(42);  
sys:print(3.14);  
sys:print(true);
```

JAGUAR

Automatically printable types:

- `string`
- `int`
- `i8`
- `u8`
- `i16`
- `u16`
- `i32`
- `u32`
- `i64`
- `u64`
- `float`
- `f32`
- `f64`

- `bool`
- certain `dynamic_list` values.

Each call prints a value followed by a newline.

14.2 Console colors

`sys:console:set_color`

```
sys:console:set_color("\\033[31m");  
sys:print("Red");  
sys:console:reset_color();
```

JAGUAR

The function expects an ANSI sequence.

`sys:console:reset_color`

```
sys:console:reset_color();
```

JAGUAR

14.3 Running a program

```
int result = sys:execute("program", ".");
```

JAGUAR

The second argument represents the working directory.

The result corresponds to the code returned by the system execution.

14.4 File system

Reading a file

```
string data = sys:fs:read("save.txt");
```

JAGUAR

Writing a file

```
sys:fs:write("save.txt", "hello");
```

JAGUAR

These functions use the C file API internally without exposing **FILE*** to the Jaguar language.

14.5 sys at a glance

FUNCTION	PURPOSE
<code>sys:print(value)</code>	Prints a value to standard output
<code>sys:console:set_color(code)</code>	Changes the console text color
<code>sys:console:reset_color()</code>	Restores the default console color
<code>sys:execute(command)</code>	Runs an external program
<code>sys:fs:read(path)</code>	Reads a file
<code>sys:fs:write(path, content)</code>	Writes a file

The `jcc` Library

`jcc` is a compact, *libc*-inspired library that ships with the compiler. It covers process control, integer helpers, math, strings, characters, files and the environment, with names chosen to be explicit about types.

TWO WAYS TO WORK WITH STRINGS

Strings offer methods (`name.length()`) and the `jcc` library offers matching functions (`jcc:string_length(name)`). Both do the same job; use whichever reads better in your code.

15.1 Process control and integer helpers

`jcc` is JaguarCC's simplified standard library.

It exposes functions inspired by *libc*, but adapted to Jaguar's types and conventions.

Pointers and overly low-level *libc* APIs are deliberately not exposed directly.

Process

`jcc:exit`

```
jcc:exit(0);
```

JAGUAR

Terminates the program with the given code.

`jcc:abort`

```
jcc:abort();
```

JAGUAR

Immediately stops the program.

Integers

jcc:abs_i32

```
i32 x = jcc:abs_i32(-42);
```

JAGUAR

jcc:min_i32

```
i32 x = jcc:min_i32(10, 20);
```

JAGUAR

jcc:max_i32

```
i32 x = jcc:max_i32(10, 20);
```

JAGUAR

jcc:clamp_i32

```
i32 x = jcc:clamp_i32(value, 0, 100);
```

JAGUAR

jcc:random_i32

```
i32 x = jcc:random_i32(0, 100);
```

JAGUAR

jcc:time_ms

```
i64 now = jcc:time_ms();
```

JAGUAR

Returns a time in milliseconds.

jcc:assert

```
jcc:assert(health >= 0, "health must not be negative");
```

JAGUAR

15.2 Math

All of the following functions primarily use `f64`.

Example:

```
f64 distance = jcc:sqrt(x * x + y * y);
f64 angle = jcc:atan2(y, x);
f64 power = jcc:pow(2.0, 8.0);
```

JAGUAR

FUNCTION	SIGNATURE
<code>sqrt</code>	<code>f64 -> f64</code>
<code>pow</code>	<code>f64, f64 -> f64</code>
<code>sin, cos, tan</code>	<code>f64 -> f64</code>
<code>asin, acos, atan</code>	<code>f64 -> f64</code>
<code>atan2</code>	<code>f64, f64 -> f64</code>
<code>floor, ceil, round</code>	<code>f64 -> f64</code>
<code>log, log10, exp</code>	<code>f64 -> f64</code>
<code>fmod</code>	<code>f64, f64 -> f64</code>

15.3 String functions

`jcc` also exposes simple wrappers around string operations.

```
string a = "Hello";
string b = " World";

int length = jcc:string_length(a);
bool same = jcc:string_equals(a, b);
string result = jcc:string_concat(a, b);
```

JAGUAR

Functions:

```
string_length
string_equals
string_compare
string_contains
string_starts_with
string_ends_with
string_concat
string_substring
```

TEXT

```
string_char_at
string_find
string_to_upper
string_to_lower
```

Example:

```
if (jcc:string_contains("Hello world", "world")) {
    sys:print("found");
}
```

JAGUAR

15.4 String conversions

String → number:

```
i32 a = jcc:string_to_i32("42");
i64 b = jcc:string_to_i64("100000");
f64 c = jcc:string_to_f64("3.14");
```

JAGUAR

Number → string:

```
string a = jcc:i32_to_string(42);
string b = jcc:i64_to_string(100000);
string c = jcc:f64_to_string(3.14);
```

JAGUAR

15.5 Character functions

Functions inspired by `ctype.h` use integer character codes.

```
bool digit = jcc:is_digit('0');
```

JAGUAR

Available functions:

```
is_digit
is_alpha
is_alnum
is_space
is_upper
is_lower
to_upper_char
to_lower_char
```

TEXT

They can be used on the result of `string.char_at()`.

Example:

```
i32 c = name.char_at(0);
if (jcc:is_alpha(c)) {
    sys:print("letter");
}
```

JAGUAR

15.6 Files and environment

`file_exists`

```
bool exists = jcc:file_exists("save.dat");
```

JAGUAR

`remove_file`

```
bool success = jcc:remove_file("save.dat");
```

JAGUAR

`rename_file`

```
bool success = jcc:rename_file("old.txt", "new.txt");
```

JAGUAR

`env_get`

```
string home = jcc:env_get("HOME");
```

JAGUAR

These functions avoid directly manipulating `FILE*`, `char*`, etc.

15.7 Reserved libc names

The compiler is aware of several libc names in order to avoid collisions with Jaguar functions.

For example, the following names are reserved when used as ordinary Jaguar functions:

```
malloc
free
memcpy
memset
printf
scanf
fopen
fclose
strlen
strcmp
strcpy
qsort
rand
srand
system
...
```

TEXT

The goal is to prevent a Jaguar function from accidentally generating a C symbol that collides with libc.

When a libc feature is useful, it should ideally be exposed through a suitable `jcc:*` Jaguar API rather than through raw C pointers.

PART IV

Interoperability and Tooling

Calling C, generating bindings, building projects, and a look inside the compiler.

IN THIS PART

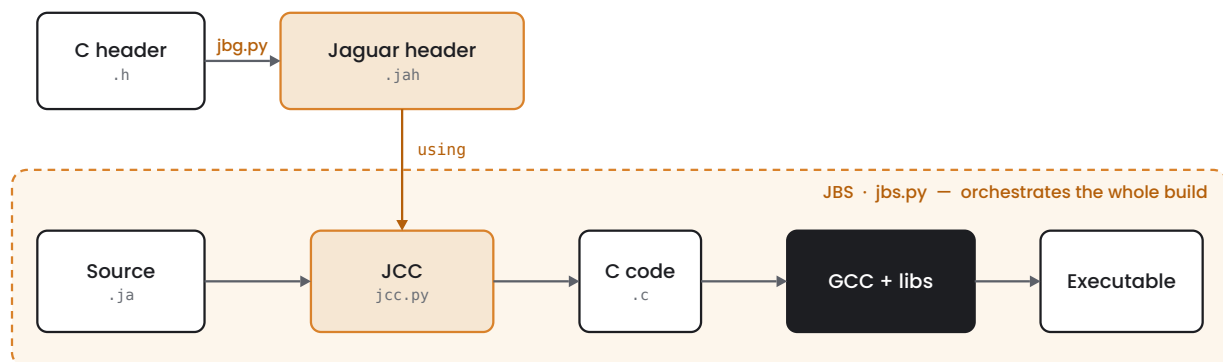
- 16** C Interoperability
- 17** Jaguar Headers and BindGen
- 18** The JBS Build System
- 19** Under the Hood: How JaguarCC Works

C Interoperability

Jaguar compiles to C, so talking to C is a first-class feature rather than an afterthought. This chapter covers the language features that make it possible: `@extern`, pointer types, `nullptr`, C strings, function pointers and ABI-preserving numeric types.

16.1 The big picture

Jaguar can call native C APIs directly. The current compiler uses Jaguar pointer types and function-pointer types rather than separate `cptr`, `cstr`, or `cfuncptr` keywords.



In practice you will mostly declare native functions in one of two ways: by hand with `@extern` for a handful of functions, or by letting BindGen (Chapter 17) generate a whole `.jah` header from a C header.

16.2 `@extern`

An external C function can be declared manually:

```
@extern
int add(int a, int b);
```

JAGUAR

`@extern` prevents Jaguar name mangling and keeps the native C symbol name.

`@extern` can also be used on a global variable:

```
@extern
u32 native_counter;
```

JAGUAR

The declaration is emitted as an external C declaration and cannot have an initializer.

16.3 Pointer types

Pointers use `*` directly on the Jaguar type:

```
i32* value;
u32* buffer;
MyStruct* object;
```

JAGUAR

Multiple pointer levels are allowed:

```
i8** strings;
void** userdata;
```

JAGUAR

Pointers are useful primarily for C interoperability and low-level APIs.

The address-of operator `&` produces a pointer:

```
u32 buffer = 0;
gl:glGenBuffers(1, &buffer);
```

JAGUAR

The unary `*` operator dereferences a pointer:

```
i32 value = 10;
i32* ptr = &value;
i32 copy = *ptr;
```

JAGUAR

Dereferencing a non-pointer is a compiler error.

16.4 nullptr

Use `nullptr` for a null pointer:

```
void* handle = nullptr;
```

JAGUAR

C-style `NULL` is not the Jaguar null-pointer literal for declarations such as:

```
void* handle = NULL; // invalid
```

JAGUAR

This distinction is especially important for global pointer initializers.

16.5 C strings

A C string such as `const char*` is represented by a pointer type, normally:

```
i8* text;
```

JAGUAR

Do not confuse this with Jaguar's native `string` type.

`string` is a Jaguar runtime object, while `i8*` is a raw C character pointer. BindGen deliberately keeps this ABI distinction.

For example, a C declaration:

```
const char* get_name(void);
```

C

is generated approximately as:

```
@extern i8* get_name();
```

JAGUAR

16.6 Function-pointer types

Jaguar supports C-compatible function-pointer types with:

```
fn(T1, T2, ...) -> ReturnType
```

TEXT

Example:

```
using Callback = fn(i32, i8*) -> void;
```

JAGUAR

A function-pointer global can be declared:

```
@extern fn() -> void callback;
```

JAGUAR

A callback can then be assigned and called:

```
void my_callback() {
    sys:print("called");
}

void main(string param) {
    callback = my_callback;
    callback();
}
```

JAGUAR

Function-pointer typedefs generated by BindGen use the same `fn(...) -> ...` syntax.

16.7 ABI-preserving numeric types

Jaguar source should use Jaguar types such as:

```
i8  u8
i16 u16
i32 u32
i64 u64
f32 f64
```

TEXT

JCC converts these to their C ABI representation when generating C. For example:

```
u32  → uint32_t
i64  → int64_t
f32  → float
```

TEXT

The C spellings such as `unsigned long long` are not Jaguar source types.

16.8 `auto` and the `=>` pointer shorthand

`auto` remains limited to variables that can be inferred from an initializer:

```
auto value = 42;
```

JAGUAR

A pointer declaration can use `=>` as an allocation-and-initialize shorthand:

```
i32* value => 42;
```

JAGUAR

This is equivalent in intent to allocating one `i32` initialized with `42`.

The shorthand is not an arbitrary aliasing operator.

16.9 `const` pointer semantics

Jaguar distinguishes a `const` pointer value from a pointer to `const` data.

Examples:

```
const i32 value = 10;
const i32* data;
i32* const ptr = &value;
```

JAGUAR

For parameters, the compiler also tracks whether the pointed-to object or the pointer itself is `const`.

The exact generated C qualifiers preserve this distinction.

16.10 Global `@extern` variables

BindGen can generate external global declarations such as:

```
@extern u32 global_counter;
```

JAGUAR

Function-pointer globals are also supported:

```
@extern fn() -> void callback;
```

JAGUAR

An `@extern` global cannot contain a Jaguar initializer.

16.11 A practical C API example

Given:

```
typedef void (*ResourceCallback)(void* resource, const void* data, size_t size);

void resource_set_callback(void* resource, ResourceCallback callback);
```

BindGen can generate Jaguar declarations using:

```
using resource;

void main(string param) {
    // use the generated API here
}
```

The generated binding preserves pointer and callback ABI information instead of converting C pointers into Jaguar `string` objects.

16.12 C to Jaguar cheat sheet

IN C	IN JAGUAR
<code>NULL</code>	<code>nullptr</code> (writing <code>NULL</code> is not valid)
<code>int *p</code>	<code>i32* p</code>
<code>char *text</code>	<code>i8* text</code> (a Jaguar <code>string</code> is not a <code>char*</code>)
<code>&x</code> and <code>*p</code>	<code>&x</code> and <code>*p</code>
<code>void (*cb)(int)</code>	<code>fn(i32) -> void</code>
<code>#include "api.h"</code>	<code>using api;</code> with a <code>.jah</code> generated by BindGen

16.13 Generated C and compiler compatibility

Jaguar generates C intended mainly for GCC and modern C compilers.

The mode:

```
--c89
```

asks the generator to produce a layout compatible with C89.

Notably, the runtime avoids using `_Bool` for `bool` and generates its own `_jBool` type.

Jaguar Headers and BindGen

A large C library can have hundreds of declarations. Writing them out by hand would be tedious and error-prone. BindGen reads a C header and writes the Jaguar equivalent for you.

17.1 Jaguar headers (.jah)

A `.jah` is a Jaguar interface file. It uses normal Jaguar syntax, but is intended to be imported and reused.

Example:

```
include/  
    math.jah  
    graphics.jah  
src/  
    main.ja
```

TEXT

A source file can request an interface with:

```
using math;  
using graphics;
```

JAGUAR

The current JBS implementation resolves `using name;` by locating either:

```
name.ja  
name.jah
```

TEXT

relative to the `.jbs` file or in an `include` directory.

Imports are recursive. A file imported more than once is only included once in the combined compilation unit.

`using` is therefore a source/module inclusion mechanism in the current JBS implementation, not a `#include`.

17.2 Generating a .jah from a C header

`jbg.py` converts a C header into a Jaguar Header (`.jah`).

Basic usage:

```
python jbg.py --c api.h
```

SHELL

By default this creates:

```
api.jah
```

TEXT

Specify an output path:

```
python jbg.py --c api.h -o include/api.jah
```

SHELL

Write the generated binding to standard output:

```
python jbg.py --c api.h --stdout
```

SHELL

Treat unsupported declarations as a hard failure:

```
python jbg.py --c api.h --strict
```

SHELL

The output file must use the `.jah` extension.

17.3 How BindGen works

The current JBG implementation parses C with `pycparser`.

Its pipeline is:

```
C header
↓
comment/preprocessor preparation
↓
pycparser AST
↓
Jaguar declaration generation
↓
.jah
```

TEXT

Simple object-like `#define` constants are preserved in the generated Jaguar header.

JBG does not blindly copy C declarations. It converts C ABI types into Jaguar types understood by the current JCC.

For example:

C	Jaguar
-----	-----
int	int / i32
unsigned int	u32
int64_t	i64
uint64_t	u64
float	f32
double	f64
const char*	i8*
void*	void*

JCC then converts the Jaguar ABI types back to the appropriate C representation during code generation.

17.4 Supported C declarations

The current JBG implementation can generate bindings for:

- function prototypes;
- `typedef` aliases;
- structs;
- unions;
- enums with sequential values;
- pointers;
- multiple pointer levels;
- function-pointer types;
- callback typedefs;
- function-pointer global variables;
- ordinary external global variables;
- forward declarations;
- nested/anonymous inline structs and unions when a stable field context can be generated;
- compatible object-like macros.

Example callback:

```
typedef void (*Callback)(void* userdata);
```

C

becomes approximately:

```
using Callback = fn(void*) -> void;
```

JAGUAR

Example function:

```
void register_callback(Callback callback);
```

C

becomes:

```
@extern void register_callback(Callback callback);
```

JAGUAR

17.5 C strings and ABI preservation

JBG deliberately does not convert `const char*` into Jaguar `string`.

For C ABI compatibility:

```
const char* name;
```

C

becomes approximately:

```
i8* name;
```

JAGUAR

Jaguar `string` is a runtime-managed Jaguar object and is therefore not an ABI-compatible replacement for a C `char*`.

17.6 Anonymous aggregate types

When a C header contains inline aggregates such as:

```

struct Resource {
    union {
        struct {
            const void* data;
            size_t size;
        } binary;
    } value;
};

```

JBG can synthesize stable Jaguar type names from the containing type and member names.

This allows the resulting `.jah` to remain expressible using Jaguar's named `struct` and `union` syntax.

17.7 Unsupported C constructs

The current JCC/JBG syntax does not represent every C declaration.

JBG currently refuses or warns about constructs including:

- C array declaration types where a Jaguar declaration cannot preserve the ABI;
- variadic functions using `...`;
- explicit enum member initializers;
- unsupported C type AST nodes;
- certain anonymous declarations that have no usable field context;
- struct fields that are C function declarations rather than supported function-pointer types.

JBG reports these as warnings when possible.

With:

```
python jbg.py --c api.h --strict
```

any generated warning causes the command to fail.

17.8 Invalid-header recovery

JBG contains a conservative recovery for one common malformed C pattern: a function-pointer declarator using a C reserved keyword as its identifier.

The recovery skips that invalid declaration and emits a warning rather than allowing the malformed declaration to prevent useful bindings from the rest of the header.

This recovery is intentionally narrow; JBG does not attempt to guess arbitrary invalid C syntax because doing so could silently create an ABI-breaking binding.

17.9 A complete example

Input C:

```
typedef struct Resource Resource;

typedef void (*ResourceCallback)(
    Resource* resource,
    const void* data,
    size_t size,
    void* userdata
);

typedef struct ResourceDescriptor {
    const char* name;
    size_t size;
    ResourceCallback callback;
} ResourceDescriptor;

Resource* resource_create(
    const ResourceDescriptor* descriptor,
    Resource** dependencies,
    size_t dependency_count
);
```

Typical generated Jaguar declarations:

```
struct Resource;

using ResourceCallback = fn(Resource*, void*, u64, void*) -> void;

struct ResourceDescriptor {
    i8* name;
    u64 size;
    ResourceCallback callback;
}

@extern Resource* resource_create(
    ResourceDescriptor* descriptor,
    Resource** dependencies,
    u64 dependency_count
);
```

The generated file can then be imported through JBS:

```
using resource;
```

JAGUAR

and the generated C calls retain the C ABI through JCC's type lowering.

The JBS Build System

The compiler translates one program at a time. JBS turns a project into finished executables and libraries: it reads a small project file, gathers the sources, expands imports, calls the compiler and links the result.

18.1 Overview

Jaguar has a separate build system: **JBS** (`jbs.py`).

The current JBS format is version `1.0`.

A `.jbs` file describes:

- the output directory;
- Jaguar source/header files belonging to each target;
- preprocessor definitions;
- include directories used to resolve `using`;
- whether C89 generation is requested;
- whether generated `.c` files should be kept;
- libraries and linker arguments;
- executable, static-library, and shared-library targets.

JBS does not currently run BindGen through a `bindgen` directive. BindGen is a separate tool described in the **Tools** section.

18.2 Minimal project

```
version 1.0
```

JBS

```
out build
```

```
compile Main {  
    main.ja  
}
```

JBS combines the listed Jaguar files, expands their `using` imports, runs JCC to generate C, then invokes the bundled GCC toolchain.

18.3 `version`

The only supported version is currently:

```
version 1.0
```

JBS

A missing version or another version is rejected.

18.4 `out`

Defines the output directory:

```
out build
```

JBS

The command-line `-o` option overrides the `out` directive:

```
python jbs.py game.jbs -o other_build
```

SHELL

18.5 `include`

Adds directories searched when resolving:

```
using graphics;
```

JAGUAR

Example:

```
include include
```

JBS

JBS searches the project directory first, then the configured include directories.

For `using graphics;`, JBS accepts:

```
graphics.ja
graphics.jah
```

TEXT

It does not treat `include` as a C compiler `-I` directive.

18.6 `define`

Adds a C preprocessor definition to the generated compilation unit:

```
define GAME_VERSION 2
define ENABLE_FEATURE
define PI 3.14
```

JBS

The resulting combined Jaguar source begins with equivalent definitions:

```
#define GAME_VERSION 2
#define ENABLE_FEATURE
#define PI 3.14
```

C

This is useful with Jaguar's supported preprocessor directives.

18.7 `c89`

The directive:

```
c89
```

JBS

asks JCC to generate C89-compatible block declaration layout.

It has the same semantic goal as:

```
python jcc.py main.ja --c89
```

SHELL

18.8 keep_c

By default, JBS removes the intermediate `.c` file after the final output has been produced.

Use:

```
keep_c
```

JBS

to keep the generated C file for inspection or debugging.

18.9 compile

Builds an executable:

```
compile Main {
  src/main.ja
  src/player.ja
  include/game.jah
}
```

JBS

The files must use `.ja` or `.jah`.

JBS performs:

```
.ja / .jah
↓
expand using
↓
JCC
↓
.c
↓
bundled GCC
↓
executable
```

TEXT

Several targets may exist in one `.jbs` file.

Without `--target`, targets are built in the order in which they appear in the project file.

18.10 `compile_static`

Builds a static library:

```
compile_static MyLib {
    src/math.ja
    src/player.ja
}
```

JBS

The normal result is:

```
libMyLib.a
```

TEXT

A static-library target does not require `main`.

JBS first generates C, compiles it to an object file, then archives the object with `ar`.

18.11 `compile_shared`

Builds a shared library:

```
compile_shared MyLib {
    src/math.ja
}
```

JBS

On Windows/MinGW the output is a DLL plus an import library:

```
MyLib.dll
MyLib.dll.a
```

TEXT

On other supported environments the generated shared-library filename follows the platform convention used by the current JBS implementation.

18.12 `link`

A target can specify libraries or linker arguments:

```
link Main {
    MyLib
    SDL2
}
```

JBS

Entries beginning with `-` are passed as linker arguments.

Examples:

```
link Main {
  -lSDL2
  -Lthird_party/lib
}
```

JBS

JBS also resolves common library names and paths, including:

```
MyLib
MyLib.a
libMyLib.a
MyLib.lib
libMyLib.so
MyLib.dll.a
```

TEXT

It searches the output directory and project directory when resolving library files.

18.13 `--target`

Build only one target:

```
python jbs.py game.jbs --target Main
```

SHELL

If the target does not exist, JBS reports an error.

Without `--target`, all targets are built in file order.

This is useful when a project defines a library before an executable that links it.

18.14 JBS command line

Current command-line forms are:

```
python jbs.py project.jbs
python jbs.py project.jbs -o build
python jbs.py project.jbs --target Main
```

SHELL

Unknown command-line options are rejected.

The project file must have the `.jbs` extension.

JBS automatically looks for `jcc.py` next to `jbs.py`.

18.15 Toolchain

The current JBS implementation resolves the bundled MinGW toolchain relative to `jbs.py`:

```
toolchain/
├─ mingw64/
│   └─ bin/
│       ├── gcc
│       └─ ar
```

TEXT

JBS therefore does not rely on a separately configured GCC in `PATH` for its normal build steps.

The exact executable suffix depends on the host platform.

18.16 `using` expansion

JBS expands imports before invoking JCC.

For:

```
using math;
```

JAGUAR

it searches for `math.ja` or `math.jah`, inserts that file's contents, and removes the `using` directive from the combined compilation unit.

Imports can themselves contain `using` directives.

Circular imports are detected and reported.

Already-loaded files are not inserted twice.

18.17 Full project example

```
MyGame/
├─ project.jbs
├─ include/
│   ├── graphics.jah
│   └─ engine.jah
├─ src/
└─ main.ja
```

TEXT

```

|   └─ game.java
└─ lib/
    └─ libMyEngine.a
    └─ build/

```

project.jbs:

```

version 1.0

out build
include include

c89
keep_c

define GAME_VERSION 1

compile Main {
    src/main.java
    src/game.java
    include/graphics.jah
}

link Main {
    MyEngine
}

```

JBS

The resulting build flow is:

```

project.jbs
  ↓
JBS
  ↓
expand .ja / .jah + using
  ↓
JCC
  ↓
Main.c
  ↓
GCC + libraries
  ↓
build/Main

```

TEXT

18.18 Recommended project organization

A typical organization can be:

```
Project/
├── build/
├── include/
│   ├── engine.jah
│   └── glad.jah
├── lib/
│   ├── libMyEngine.a
│   └── ...
├── src/
│   ├── main.ja
│   └── game.ja
└── project.jbs
```

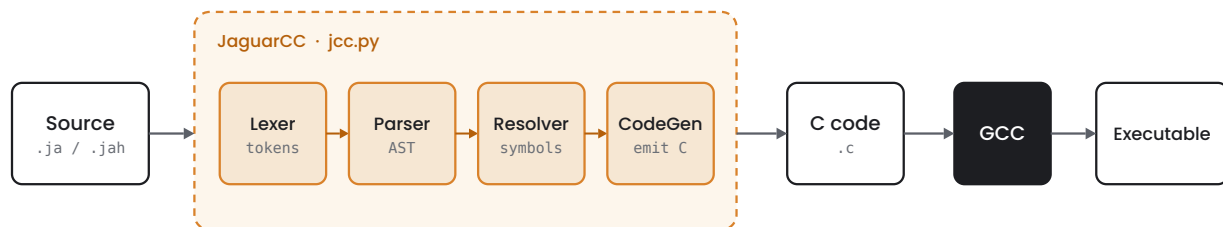
TEXT

Reusable interfaces are placed in `.jah` files, implementations in `.ja` files, and external C APIs are generally introduced via `jbg.py`.

Under the Hood: How JaguarCC Works

You can use Jaguar without knowing how the compiler works, but a little knowledge goes a long way when you read generated C, debug a linker error or write an editor plugin.

19.1 From source to executable



JaguarCC is a Python program (`jcc.py`) organized as four stages, followed by the C compiler:

1. The **lexer** reads the source text and produces tokens. Comments, including documentation comments written `/** ... */`, are removed here.
2. The **parser** builds a syntax tree and keeps track of source lines in its nodes, so that diagnostics can point to the right place.
3. The **resolver** looks up names, chooses function overloads, applies the allowed numeric conversions and checks access control.
4. The **code generator** emits C, plus the small runtime that backs strings, classes, containers, lists, maps and dynamic lists.

GCC then compiles the C, and JBS (when you use it) links in your libraries.

19.2 What ends up in the C

The mapping from Jaguar to C follows a few simple rules. Knowing them makes generated code much easier to read.

JAGUAR	IN THE GENERATED C
<code>void main(string param)</code>	A standard C <code>main</code> (never a non-standard <code>main(string)</code>)
<code>bool</code>	The runtime's own <code>_jBool</code> type instead of <code>_Bool</code>
<code>struct</code>	A C <code>typedef struct</code>
A method	A function with an implicit <code>self</code> parameter
<code>math:add</code> (namespaced function)	A mangled name, for example <code>math_add</code>
An overloaded function	A name that also includes the parameter types
<code>@extern</code> function	The exact C symbol name, unmangled

19.3 Name mangling

Jaguar generates C names compatible with namespaces and overloading.

A simple function:

```
int add(int a, int b) {
    return a + b;
}
```

JAGUAR

generally stays `add` on the C side when it is not overloaded.

A function inside a namespace:

```
namespace math {
    int add(int a, int b) {
        return a + b;
    }
}
```

JAGUAR

is mangled to avoid collisions, for example in a form like:

```
math_add
```

TEXT

An overload also uses the parameter types in its C name.

19.4 Diagnostics and source locations

JCC tracks source lines in parser AST nodes. Parser errors include the relevant line in their diagnostics.

Semantic/code-generation diagnostics also use the best available source line when the exact AST location is not directly available.

This is especially useful to editor integrations and the Jaguar language server.

19.5 One unit, many files

The compiler itself sees a single compilation unit. When a project has several files, JBS resolves every `using name;` by locating `name.ja` or `name.jah` relative to the project file or in an `include` directory, and combines them into one unit before calling the compiler. An imported file is included only once, however many files import it.

19.6 Tool responsibilities

The tools intentionally have separate responsibilities:

```
jbg.py
C .h
↓
Jaguar .jah

jcc.py
Jaguar .ja/.jah
↓
C
↓
executable/output

jbs.py
project .jbs
↓
combines .ja/.jah
↓
JCC
↓
bundled GCC
↓
final project outputs
```

TEXT

JBG is therefore a binding generator, JCC is the compiler, and JBS is the project build orchestrator.

NOTE

Call resolution, the rules that pick one overload among several, is described with functions in Chapter 7.

PART V

Guides and Cookbook

Hands-on tutorials, ready-to-use recipes, a comparison with C and C++, and a troubleshooting checklist.

IN THIS PART

- 20** Guide: A Temperature Table
- 21** Guide: Modeling a Game with Classes
- 22** Guide: A Scoreboard with Collections
- 23** Guide: A Multi-File Project with JBS
- 24** Guide: Calling C from Jaguar
- 25** Recipes
- 26** Coming from C and C++
- 27** Pitfalls and Troubleshooting

Guide: A Temperature Table

Your first real program: a Celsius-to-Fahrenheit conversion table. It is small enough to write in ten minutes and it exercises functions, casts, loops, numeric types and the standard library.

20.1 What we are building

The program prints one line per step, from 0 to 100 degrees Celsius in steps of ten, showing the matching Fahrenheit value. Along the way you will practice:

- writing a function with a decimal parameter and result;
- the inclusive `for_loop`;
- explicit C-style casts;
- turning numbers into strings with the `jcc` library.

20.2 Step 1: the conversion function

Start with the arithmetic. A function takes a Celsius value and returns Fahrenheit.

```
f64 to_fahrenheit(f64 celsius) {  
    return celsius * 9.0 / 5.0 + 32.0;  
}
```

JAGUAR

Notice that the literals are written `9.0`, `5.0` and `32.0`. Decimal literals are `f64`, so the whole expression stays in floating point.

20.3 Step 2: the loop

`for_loop(0, 10)` walks from 0 to 10 **included**, so it produces the eleven steps we need. The index is an `int` and it is read-only inside the loop.

```
void main(string param) {
    int step = for_loop(0, 10) {
        f64 celsius = (f64)(step * 10);
        f64 fahrenheit = to_fahrenheit(celsius);
        sys:print(fahrenheit);
    }
}
```

JAGUAR

The cast `(f64)(step * 10)` converts the integer to a decimal. Jaguar only accepts C cast syntax, so `f64(step * 10)` would not compile.

20.4 Step 3: readable output

Printing bare numbers is not very friendly. The `jcc` library converts numbers to strings and joins strings.

```
f64 to_fahrenheit(f64 celsius) {
    return celsius * 9.0 / 5.0 + 32.0;
}

void main(string param) {
    int step = for_loop(0, 10) {
        f64 celsius = (f64)(step * 10);
        f64 fahrenheit = to_fahrenheit(celsius);

        string left = jcc:string_concat(jcc:f64_to_string(celsius), " C = ");
        string right = jcc:string_concat(jcc:f64_to_string(fahrenheit), " F");
        sys:print(jcc:string_concat(left, right));
    }
}
```

JAGUAR

20.5 Step 4: compile and run

Save the file as `temperatures.ja`, then:

```
python jcc.py temperatures.ja -o temperatures
```

SHELL

Running the program prints eleven lines, one for each step.

WHY SO MANY VARIABLES?

Each `string_concat` call joins exactly two strings, so building a longer message takes a few steps. Giving each intermediate result a name keeps the code easy to read.

20.6 What you practiced

- Functions with typed parameters and a return value (Chapter 7).
- The inclusive `for_loop` and its read-only index (Chapter 8).
- C-style casts (Chapter 5).
- The `jcc` conversion and string functions (Chapter 15).

EXTEND IT

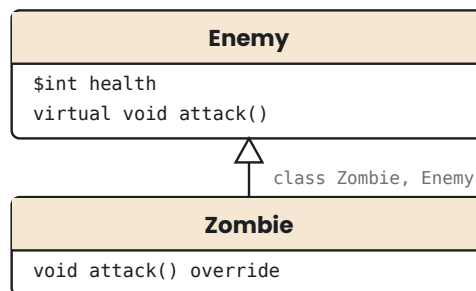
Turn the table into a two-way converter by adding a second function, `to_celsius`, and print both columns. Then try changing the step size: what happens to the number of lines when you change the last bound of `for_loop`?

Guide: Modeling a Game with Classes

Classes shine when a program has things that have state and behavior. In this guide you will build a tiny turn-based fight between a player and a zombie, using inheritance and a virtual method.

21.1 Design first

We need three kinds of objects. A **Player** has health and can take damage. An **Enemy** is a generic opponent with health and an **attack** behavior that can be replaced. A **Zombie** is a particular kind of enemy that attacks in its own way.



21.2 Step 1: the player

The player has a public **health** field, a method that reduces it, and a **const** method that answers a question without changing anything.

```

class Player {
    $int health = 100;

    void damage(int amount) {
        health = health - amount;
    }
}
  
```

JAGUAR

```

    bool alive() const {
        return health > 0;
    }
}

```

21.3 Step 2: a base class with a virtual method

`Enemy` declares `attack` as `virtual`, which allows derived classes to replace it. Its `health` field is public, so the code in `main` can read it as well as the derived classes.

```

class Enemy {
    $int health;

    virtual void attack() {
        sys:print("The enemy attacks!");
    }
}

```

JAGUAR

21.4 Step 3: a derived class

A derived class names its base after a comma. `override` marks the replacement, and it must match the virtual method's signature exactly. The derived class can read and write the inherited `health` field.

```

class Zombie, Enemy {
    constr(int start_health) {
        health = start_health;
    }

    void attack() override {
        sys:print("The zombie claws at you!");
    }

    void hurt(int amount) {
        health = health - amount;
    }

    bool alive() const {
        return health > 0;
    }
}

```

JAGUAR

21.5 Step 4: the fight

The fight is a loop with no condition. Each turn, the zombie attacks, both sides lose health, and we test for a winner. `break` leaves the loop.

```
void main(string param) {
    Player player = Player();
    Zombie zombie = Zombie(60);

    int turn = for_loop(1, 20) {
        zombie.attack();
        player.damage(jcc:random_i32(5, 15));
        zombie.hurt(jcc:random_i32(10, 20));

        sys:print(jcc:string_concat("Player health: ", jcc:i32_to_string(player.health)));
        sys:print(jcc:string_concat("Zombie health: ", jcc:i32_to_string(zombie.health)));

        if (!player.alive()) {
            sys:print("You were defeated.");
            break;
        }
        if (!zombie.alive()) {
            sys:print("The zombie is destroyed!");
            break;
        }
    }
}
```

The loop is bounded by `for_loop(1, 20)` rather than by `while`, which guarantees that the fight ends even if nobody wins. Each turn also uses `jcc:random_i32` to vary the damage.

21.6 What you practiced

- Classes with fields, methods and `const` methods (Chapter 10).
- Access control with `$` markers.
- Inheritance, `virtual` and `override`.
- Using `break` inside a loop, and the standard library for random numbers and string building.

EXTEND IT

Add a second enemy type, `Skeleton`, that overrides `attack` differently. Give `Player` a constructor that takes a starting health value, using a default parameter so that `Player()` still works.

Guide: A Scoreboard with Collections

Collections are the everyday data structures of a Jaguar program. In this guide you will build a small scoreboard with a `list`, a `map` and both of the collection loops, and discover the one rule that catches most newcomers: collections are changed with methods, never by indexing.

22.1 Step 1: a list of scores

A `list<int>` is created with braces and grows with `push`.

```
void main(string param) {  
    list<int> scores = {30, 45, 12};  
    scores.push(58);  
  
    int first = scores[0];  
    sys::print(first);  
}
```

JAGUAR

Reading with `scores[0]` is fine. Writing with `scores[0] = 99;` is not allowed, because indexing is read-only by design.

22.2 Step 2: walking the list

`loop_list` visits every element in order. The loop variable, declared with `auto`, receives each value in turn. Here we sum the scores in a variable declared **before** the loop.

```
void main(string param) {  
    list<int> scores = {30, 45, 12};  
    scores.push(58);  
  
    int total = 0;  
    auto score = loop_list(scores) {
```

JAGUAR

```

        total = total + score;
    }

    sys::print(jcc::string_concat("Total: ", jcc::i32_to_string(total)));
}

```

During the loop the list is read-only: calling `scores.push(...)` inside `loop_list` is forbidden. If you need to add elements based on what you see, collect them first, and add them afterwards.

22.3 Step 3: names and scores in a map

A `map<string, int>` associates a name with a score. Braces list the entries as alternating keys and values, and `emplace` adds or replaces an entry.

```

void main(string param) {
    map<string, int> board = {
        "alice", 30,
        "bob", 45
    };
    board.emplace("charlie", 12);
    board.emplace("alice", 35);    // replaces alice's score

    int alice = board["alice"];
    sys::print(alice);
}

```

JAGUAR

String keys are compared by their content, so `board["alice"]` finds the entry no matter where the key string came from.

22.4 Step 4: walking the map

`loop_map` gives you a `pair<string, int>` for every entry. The pair has two members, `first` and `second`.

```

void main(string param) {
    map<string, int> board = {
        "alice", 30,
        "bob", 45
    };
    board.emplace("charlie", 12);

    int best = 0;
    auto entry = loop_map(board) {
        string line = jcc::string_concat(entry.first, ": ");
    }
}

```

JAGUAR

```

        sys:print(jcc:string_concat(line, jcc:i32_to_string(entry.second)));
        best = jcc:max_i32(best, entry.second);
    }

    sys:print(jcc:string_concat("Best: ", jcc:i32_to_string(best)));
}

```

22.5 Step 5: mixed data

Sometimes a value does not have one type. A `dynamic_list` stores values of several types together, and casts recover them.

```

void main(string param) {
    dynamic_list record = {"alice", 30, true};

    string kind = record.type(1);
    sys:print(kind);

    i32 score = (i32)record[1];
    sys:print(score);
}

```

JAGUAR

22.6 What you practiced

- `list`, `map`, `pair` and `dynamic_list` (Chapter 11).
- Literals with braces, `push` and `emplace`.
- `loop_list` and `loop_map`, and their read-only rule.
- Read-only indexing.

COLLECTIONS ARE LOCAL VARIABLES

In the current implementation, collection types such as `list` and `map` cannot be global variables. Declare them inside a function and pass them where they are needed.

Guide: A Multi-File Project with JBS

One file is enough for a tutorial, but real programs are split up. In this guide you will move a piece of code into a Jaguar header, import it with `using`, and build the whole thing with JBS.

23.1 The layout

Create a project directory with this structure:

```
Geometry/  
├── include/  
│   └── geometry.jah  
├── src/  
│   └── main.ja  
└── project.jbs
```

TEXT

Reusable declarations go in `.jah` files under `include/`. The program itself stays in `src/`.

23.2 Step 1: the reusable code

A `.jah` file uses ordinary Jaguar syntax. Put a small namespace in `include/geometry.jah`:

```
namespace geometry {  
    f64 area(f64 width, f64 height) {  
        return width * height;  
    }  
  
    f64 perimeter(f64 width, f64 height) {  
        return 2.0 * (width + height);  
    }  
}
```

JAGUAR

23.3 Step 2: the program

`src/main.ja` asks for the header with `using`, then calls the functions through their namespace.

```
using geometry;

void main(string param) {
    f64 a = geometry:area(3.0, 4.5);
    f64 p = geometry:perimeter(3.0, 4.5);

    sys:print(a);
    sys:print(p);
}
```

JAGUAR

23.4 Step 3: the project file

The `.jbs` file tells JBS where to write the output, where to look for imports, and what to build.

```
version 1.0

out build
include include

compile Main {
    src/main.ja
}
```

JBS

`include include` adds the `include/` directory to the places where JBS looks when it resolves `using geometry`; For that line, JBS accepts `geometry.ja` or `geometry.jah`.

23.5 Step 4: build

```
python jbs.py project.jbs
```

SHELL

JBS expands the `using` line by inserting the contents of `geometry.jah` into the combined compilation unit, runs JCC to produce C, and calls the bundled GCC toolchain. The executable appears in `build/` under the target name, `Main`.

23.6 Step 5: look inside

Add one line to `project.jbs` and rebuild to keep the generated C file after the build:

```
keep_c
```

JBS

Open it and find the namespaced functions. Because they live in a namespace, their C names are mangled, in a form like `geometry_area`.

23.7 Step 6: configure the build

JBS directives can be combined freely. For example, this project file requests C89-compatible output, keeps the C, and passes a macro that your code can test with `#ifdef`:

```
version 1.0

out build
include include

c89
keep_c
define GEOMETRY_DEBUG 1

compile Main {
    src/main.java
}
```

JBS

23.8 What you practiced

- A project layout with `include/` and `src/` (Chapter 18).
- Jaguar headers and `using` (Chapter 17 and Chapter 12).
- The `version`, `out`, `include`, `compile`, `keep_c`, `c89` and `define` directives.

BUILD ONE TARGET AT A TIME

When a project defines several targets, `python jbs.py project.jbs --target Main` builds only the one you name.

Guide: Calling C from Jaguar

Most useful libraries are written in C. This guide shows the two ways to call them: declaring a function by hand with `@extern`, and generating a whole Jaguar header with BindGen.

24.1 The C side

We will call a tiny C library. Create `mathx.h` and `mathx.c`:

```
/* mathx.h */  
int add(int a, int b);
```

```
/* mathx.c */  
#include "mathx.h"  
  
int add(int a, int b) {  
    return a + b;  
}
```

Compile it into a static library with the usual tools:

```
gcc -c mathx.c  
ar rcs libmathx.a mathx.o
```

24.2 Way 1: declare the function by hand

For one or two functions, `@extern` is enough. It tells JCC that the function is defined elsewhere and that its C name must be kept exactly as written.

```

@extern
int add(int a, int b);

void main(string param) {
    int result = add(2, 3);
    sys:print(result);
}

```

JAGUAR

Now tell JBS to link the library. A `link` entry can be a plain library name, and JBS resolves common forms such as `libmathx.a` by searching the output directory and the project directory.

```

version 1.0

out build

compile Main {
    main.ja
}

link Main {
    mathx
}

```

JBS

```
python jbs.py project.jbs
```

SHELL

24.3 Way 2: generate the header with BindGen

For a real library with dozens of functions, structs and callbacks, write nothing by hand. Ask BindGen to turn the C header into a Jaguar header:

```
python jbg.py --c mathx.h -o include/mathx.jah
```

SHELL

BindGen parses the header, converts each C declaration to its Jaguar equivalent and writes an `.jah` file. Open it to see what was generated: you will find `@extern` declarations, plus structs, aliases and constants for anything else the header declared.

Then import the header in your program and drop the hand-written declaration:


```
using mathx;

void main(string param) {
    int result = add(2, 3);
    sys:print(result);
}
```

JAGUAR

And tell JBS where to find the generated header:

```
version 1.0

out build
include include

compile Main {
    main.ja
}

link Main {
    mathx
}
```

JBS

24.4 Things to remember about C types

- A C string (`const char*`) becomes `i8*`, not Jaguar's `string`. BindGen keeps this distinction on purpose.
- Pointers are written with `*` after the type, the address-of operator is `&`, and the null pointer is `nullptr`.
- A C function-pointer type becomes `fn(...) -> ...`.
- The C `int` maps to `int` (or `i32`), `unsigned int` to `u32`, and so on. The table in Chapter 17 lists the full mapping.

STRICT MODE

BindGen reports unsupported declarations as warnings when it can. Add `--strict` to the `jbg.py` command so that any warning makes the command fail. It is a good default for automated builds.

24.5 What you practiced

- `@extern` declarations (Chapter 16).
- Linking with a static library through JBS (Chapter 18).
- Generating a `.jah` file with BindGen (Chapter 17).

Recipes

Short, self-contained answers to questions that come up again and again. Each recipe uses only features documented in this book.

25.1 Walk over the characters of a string

`for_loop` includes its last bound, so stop at `length() - 1`. `char_at` returns the character code as an `i32`. If the string is empty, the loop starts after it ends and does nothing.

```
string word = "Jaguar";
int last = word.length() - 1;

int i = for_loop(0, last) {
    sys:print(word.char_at(i));
}
```

JAGUAR

25.2 Count the digits in a string

The character functions in `jcc` take integer character codes, which is exactly what `char_at` returns.

```
string text = "room 101";
int digits = 0;

int i = for_loop(0, text.length() - 1) {
    if (jcc:is_digit(text.char_at(i))) {
        digits = digits + 1;
    }
}
sys:print(digits);
```

JAGUAR

25.3 Keep a value inside a range

```
int health = 90;
int healed = jcc:clamp_i32(health + 30, 0, 100); // 100
```

JAGUAR

25.4 Roll a die

```
i32 roll = jcc:random_i32(1, 6);
sys:print(roll);
```

JAGUAR

25.5 Time a piece of code

`jcc:time_ms` returns milliseconds as an `i64`.

```
i64 start = jcc:time_ms();

// ... the work you want to measure ...

i64 elapsed = jcc:time_ms() - start;
sys:print(jcc:string_concat("Elapsed ms: ", jcc:i64_to_string(elapsed)));
```

JAGUAR

25.6 Convert between text and numbers

```
i32 count = jcc:string_to_i32("42");
f64 ratio = jcc:string_to_f64("3.14");
string label = jcc:i32_to_string(count);
```

JAGUAR

25.7 Compare strings by content

Use `equals` (or `jcc:string_equals`) for the contents of two strings, and `contains`, `starts_with` and `ends_with` for partial matches.

```
string name = "Jaguar";

if (name.equals("Jaguar")) {
    sys:print("exact match");
}
if (name.starts_with("Jag")) {
    sys:print("prefix match");
}
```

JAGUAR

25.8 Color the console output

The color function expects an ANSI escape sequence, and the reset function restores the default.

```
sys:console:set_color("\\033[31m");
sys:print("Red");
sys:console:reset_color();
```

JAGUAR

25.9 Read or create a save file

```
if (jcc:file_exists("save.txt")) {
    string data = sys:fs:read("save.txt");
    sys:print(data);
} else {
    sys:fs:write("save.txt", "level=1");
}
```

JAGUAR

25.10 Read an environment variable

```
string home = jcc:env_get("HOME");
sys:print(home);
```

JAGUAR

25.11 Stop early with an error code

```
if (!jcc:file_exists("config.txt")) {
    sys:print("missing configuration");
    jcc:exit(1);
}
```

JAGUAR

25.12 Check an assumption while developing

```
int health = 10;
jcc:assert(health >= 0, "health must not be negative");
```

JAGUAR

25.13 Enable code only in debug builds

Define a macro in the JBS project file, then test it in the source.

```
define DEBUG 1
```

JBS

```
#ifdef DEBUG
    sys:print("debug build");
#endif
```

JAGUAR

25.14 Give a constructor a default

Default parameters work on constructors, so `Player()` and `Player(50)` are both valid.

```
#define MAX_HEALTH 100

class Player {
    $int health = MAX_HEALTH;

    constr(int initial_health = MAX_HEALTH) {
        health = initial_health;
    }
}
```

JAGUAR

25.15 Run another program

`sys:execute` takes the program and a working directory, and returns the exit code.

```
int code = sys:execute("program", ".");  
sys:print(code);
```

JAGUAR

Coming from C and C++

If you write C or C++, most of Jaguar will feel immediately familiar, and a handful of things will trip you up until you have seen them once. This chapter puts them all in one place.

26.1 What stays the same

- Braces for blocks, semicolons for statements, and explicit types on every declaration.
- C operators and C-like precedence, and **C cast syntax**: `(f64)value`.
- `#define`, `#if`, `#ifdef` and friends, passed through to the generated C.
- Structs, enums, unions and function pointers, with C-compatible layouts.
- Fixed-width integers, spelled `i8` to `i64` and `u8` to `u64`.
- Compilation to native code through GCC, so performance is that of C.

26.2 What is different

TOPIC	C / C++	JAGUAR
<code>while</code> loop	<code>while (x > 0) { ... }</code>	<code>while { ... }</code> with an explicit <code>break</code>
Counting loop	<code>for (int i = 0; i <= 10; i++)</code>	<code>int i = for_loop(0, 10) { ... }</code> (last bound included , index read-only)
Namespaces	<code>math::add(1, 2)</code>	<code>math:add(1, 2)</code>
Null pointer	<code>NULL</code> or <code>nullptr</code>	<code>nullptr</code> only
C string	<code>const char*</code>	<code>i8*</code> ; Jaguar's <code>string</code> is a runtime object
Including a header	<code>#include "api.h"</code>	<code>using api;</code> with a <code>.jah</code> from BindGen
Public / protected	<code>public:</code> / <code>protected:</code> sections	<code>\$</code> and <code>%</code> markers on each member
Constructors	<code>Player(int h)</code>	<code>constr(int h)</code>
Destructors	<code>~Player()</code>	<code>destr()</code>

TOPIC	C / C++	JAGUAR
Inheritance	<code>class Zombie : public Enemy</code>	<code>class Zombie, Enemy</code>
Generic containers	Templates (<code>std::vector<int></code>)	Built-in <code>list<T></code> , <code>map<K,V></code> , <code>pair<K,V></code>
Element assignment	<code>v[2] = 10;</code>	Not allowed; use methods such as <code>push</code> and <code>emplace</code>
Casting	<code>static_cast<int>(x)</code>	<code>(int)x</code>
Memory	<code>malloc</code> / <code>free</code> , <code>new</code> / <code>delete</code>	Managed by the runtime; <code>container<T></code> owns its object
Boolean type	<code>bool</code> (<code>_Bool</code> in C99)	<code>bool</code> , generated as the runtime's <code>_jBool</code>
Default arguments	C++ only	Built in, plus named arguments

26.3 What is missing on purpose

Several C and C++ features are deliberately not part of the language. You will not find:

- general pointer arithmetic;
- `malloc`, `free`, `memcpy` and other libc memory functions as a general API;
- `FILE*`; file access goes through `sys:fs` and `jcc`;
- C++ templates and other complex C++ features;
- lambdas;
- exceptions;
- multiple inheritance.

WHY LEAVE THEM OUT?

Every item on this list is either a major source of bugs (raw memory management) or a large source of language complexity (templates, exceptions). Jaguar keeps a small core and offers controlled escape hatches: pointer types, `@extern` and `.jah` bindings are there when you need to talk to C.

26.4 A quick translation

Here is a small C++ snippet and its Jaguar equivalent.

TEXT

```
// C++
class Player {
public:
    int health = 100;
    Player(int h) : health(h) {}
    void damage(int amount) { health -= amount; }
    bool alive() const { return health > 0; }
};
```

JAGUAR

```
// Jaguar
class Player {
    $int health = 100;

    constr(int h) {
        health = h;
    }

    void damage(int amount) {
        health = health - amount;
    }

    bool alive() const {
        return health > 0;
    }
}
```

COMPOUND ASSIGNMENT

Every example in the reference writes updates in the long form, `health = health - amount;`. This book does the same.

Pitfalls and Troubleshooting

Most compile errors in Jaguar come from a small set of mistakes, and nearly all of them come from expecting C behavior where Jaguar deliberately does something else. Here are the common ones, each with the fix.

27.1 Loops

while with a condition

```
while (health > 0) {    // invalid
}
```

JAGUAR

while takes no condition. Test inside the loop and leave with **break**:

```
while {
    if (health <= 0) {
        break;
    }
    health = health - 1;
}
```

JAGUAR

One iteration too many

for_loop(0, list_length) includes **list_length** itself. To visit indexes 0 to **length - 1**, stop at **length - 1**.

Changing a collection inside its loop

```
auto item = loop_list(numbers) {
    numbers.push(10);    // forbidden
}
```

JAGUAR

Both `loop_list` and `loop_map` treat the collection as read-only while the loop runs. Collect what you need to add, and add it after the loop.

27.2 Collections

Assigning through an index

```
numbers[2] = 10;           // invalid
scores["alice"] = 100;     // invalid
```

JAGUAR

Indexing only reads. Use the collection's methods, such as `push` for a list and `emplace` for a map.

Collections as globals

`list`, `map`, `pair`, `container` and `dynamic_list` are not currently valid as global variables. Declare them as locals.

Indexing a container

`container<T>` values cannot be indexed with `[]`. Use `get()` to read the value it owns.

27.3 Types and casts

Function-style casts

```
i32(3.14);                // not a cast
```

JAGUAR

Jaguar only accepts C-style casts: `(i32)3.14`.

NULL instead of `nullptr`

```
void* handle = NULL;       // invalid
void* handle = nullptr;    // valid
```

JAGUAR

auto without an initializer

```
auto x;           // invalid: nothing to deduce from
auto x = 42;      // valid
```

JAGUAR

A Jaguar string where C expects char*

A `string` is a runtime object and a C string is an `i8*`. They are different types on purpose, and BindGen keeps declarations from C headers in terms of `i8*`.

27.4 Functions

A positional argument after a named one

```
move(x = 10, 20);      // invalid
move(x = 10, y = 20);  // valid
```

JAGUAR

Named arguments to built-in functions

`jcc:*` and `sys:*` functions do not accept named arguments. Pass their arguments by position.

Overloads that are equally good

When several overloads remain equally valid after the numeric conversions are applied, the call is considered ambiguous. Cast an argument, or rename one of the functions.

Reserved libc names

Names such as `malloc`, `free`, `printf`, `strlen`, `rand` and `system` are reserved when used as ordinary Jaguar function names, to avoid collisions with libc symbols. Choose another name, or reach the feature through a `jcc:*` function.

27.5 Classes

Mismatched `override`

`override` must match a `virtual` method of the base class, with an **exact** signature. A small difference in parameter types is enough to make it fail.

Exposing a non-public field

`@exposed` can only be used on a public (\$) field, and `@register` on a class. An unknown attribute is a compile error.

27.6 Scope and globals

Shadowing

Two sibling blocks can reuse a name, but a block cannot redeclare a variable that is visible from an enclosing scope.

Non-constant global initializers

```
int x = get_value();    // invalid at global scope
```

JAGUAR

A global initializer must be a literal, a macro or an operation between constants.

Variables in a namespace

Namespaces support functions, structs, classes, enums and nested namespaces, but a global variable directly inside a namespace is not supported.

27.7 Enums and interop

Explicit enum values

```
enum Color {
    Red = 10           // invalid
}
```

JAGUAR

Enums are numbered sequentially and explicit initializers are not accepted. BindGen refuses headers that use them.

Initializing an `@extern` global

An `@extern` global declaration cannot have an initializer. It only announces a variable that lives in C.

27.8 Build and tooling

`#include`

```
#include <stdio.h>           // invalid
```

JAGUAR

Generate a `.jah` with BindGen and import it with `using`.

An output name ending in `.c`

`python jcc.py main.ja -o game.c` is wrong: `-o` takes an output base name. Use `-o game`.

An unknown JBS option

JBS rejects unknown command-line options, and the project file must have the `.jbs` extension.

WHEN THE MESSAGE IS UNCLEAR

Errors from the parser include the relevant source line. If a semantic error seems to point to the wrong place, look at the declaration a few lines above, and generate the C with `python jcc.py file.ja > file.c` to see how the compiler understood it.

PART VI

History and Roadmap

Where Jaguar comes from, the decisions that shaped it, and a proposed plan for what comes next.

IN THIS PART

- 28** The Story of Jaguar
- 29** A Development Plan

The Story of Jaguar

Every language is a collection of decisions. This chapter tells the story of Jaguar through the decisions its documentation records: what the language chose to do, what it chose not to do, and how its tools came to be separate.

ABOUT THIS CHAPTER

This history is reconstructed from the language documentation and from the current implementation. It concentrates on decisions and capabilities rather than dates: the source material describes what Jaguar is, and does not record when each part was added.

28.1 The starting idea: a language that compiles to C

The central idea of Jaguar, as its documentation presents it, is to keep the performance and reach of C while smoothing many of its sharp edges. The mechanism is a **source-to-source compiler**: a Jaguar program is translated to C by JaguarCC, and the C is compiled by GCC.

That single choice helps explain much of the rest. Because the output is C, Jaguar programs run at native speed, can be built almost anywhere a C compiler exists (down to a C89 mode for older toolchains) and can call any C library. Because the front end is its own language, it can offer things that C cannot: classes, namespaces, overloading, default and named arguments, built-in collections and reflection.

The language's stated goals capture the balance: ease of use, syntax close to C and C++, the performance of a compiled language, explicit type control, practical modern features, a minimal runtime and interoperability with C.

28.2 A compiler in four stages

JaguarCC is organized as a classic pipeline: a lexer, a parser, a resolver and a code generator. Each stage owns one kind of decision. The lexer decides what the tokens are, the parser decides what the program's structure is, the resolver decides what every name and call means, and the code generator decides what C to write.

The pipeline is worth noticing for a second reason: because the parser records source lines in its syntax tree, the compiler can report **where** a problem is. The documentation explicitly mentions editor integrations and a language server as consumers of that information.

28.3 The signature decisions

The documentation singles out a set of choices as "particularly specific to the current implementation". They are the closest thing Jaguar has to a personality:

DECISION	WHAT IT LOOKS LIKE	THE IDEA BEHIND IT
Conditionless <code>while</code>	<code>while { ... break; }</code>	The exit test is written where it happens, and loops that can end have a visible <code>break</code>
Inclusive <code>for_loop</code>	<code>for_loop(0, 10)</code> runs 0 to 10	A counting loop that says what it counts, with a read-only index
Collections without templates	<code>list<T></code> , <code>map<K,V></code> built in	Common containers are part of the language and its runtime, not a library puzzle
Read-only iteration	No changes inside <code>loop_list</code> / <code>loop_map</code>	Rules out a whole family of iterator bugs
Read-only indexing	<code>numbers[2] = 10;</code> is invalid	Writes go through named methods, so they are easy to find
<code>container<T></code>	<code>container<Player> p = new Player(100);</code>	Ownership is explicit, and the container destroys what it owns
Attribute-driven reflection	<code>@register</code> , <code>@exposed</code>	Dynamic access exists, but only where you opt in
C89 generation strategy	<code>--c89</code> , <code>_jBool</code>	Portability down to very old C compilers

28.4 How C interoperability grew up

The documentation tells the interoperability story in layers, and it is instructive to read them in order.

Declare. The simplest layer is `@extern`: a function or variable is defined elsewhere, and its C name is preserved exactly.

Unify. The current compiler expresses C types with Jaguar's own type syntax (`T*`, `T**`, `fn(...) -> R`, `nullptr`) rather than with separate special-purpose keywords such as `cptr`, `cstr` or `cfuncptr`. That is a deliberate simplification: pointers and callbacks look like the rest of the language.

Preserve. Along the way, a rule appears that matters more than any syntax: a C string is `i8*`, never Jaguar's runtime `string`, and numeric types map to ABI-exact Jaguar types. The bindings keep the C ABI intact, and the compiler lowers types back to C when it generates code.

Generate. Finally, a tool takes over the boring work. BindGen reads a C header with `pycparser` and writes a Jaguar header that is imported with `using`. It is honest about its limits: it refuses or warns about arrays it cannot represent, variadic functions and explicit enum initializers, and its recovery for malformed headers is intentionally narrow, because guessing at arbitrary invalid C could silently create an ABI-breaking binding.

The ban on `#include` completes the picture. C headers do not enter Jaguar through the preprocessor; they enter through bindings that the compiler understands.

28.5 From one compiler to a toolchain

The last chapter of the story is about structure. Jaguar's tooling is deliberately split, and each tool has one responsibility:

- **JaguarCC** translates Jaguar to C and can call GCC directly for a quick build.
- **JBS** orchestrates projects: it reads a small project file, expands `using` imports, calls the compiler with the right options and links libraries, with a bundled toolchain so builds do not depend on the machine's configuration.
- **BindGen** turns C headers into Jaguar headers.

The split lets each tool evolve independently of the others. Notably, the build system does not run BindGen by itself yet; the tools cooperate through files.

28.6 The principles behind the choices

Reading the decisions together, a handful of principles emerge.

1. **Make the common thing short and the dangerous thing explicit.** Types, casts, pointers and `nullptr` are all visible in the source.
2. **Prefer boundaries to breadth.** No exceptions, lambdas, templates, multiple inheritance or raw memory API: the language stays small on purpose.
3. **Preserve the C ABI, never guess.** Interoperability is exact or it is refused.
4. **One tool, one job.** The compiler, the build system and the binding generator meet at file boundaries.
5. **Be honest about the implementation.** The documentation says plainly that it describes the compiler's current behavior, and that anything unspecified is decided by `jcc.py`.

WHERE THE STORY GOES NEXT

A language whose documentation is this candid about its limits has a natural roadmap: the limits themselves. The next chapter turns them into a development plan.

A Development Plan

A language grows best in the direction its limits point. This chapter takes the boundaries and limitations that the documentation states plainly and arranges them into a plan: what to protect, what to finish and what to build next.

A PROPOSAL, NOT A PROMISE

This plan is derived from the limitations and boundaries recorded in the documentation. It is offered as a starting point for discussion. Priorities and phases are recommendations, meant to be adjusted by the people who build Jaguar.

29.1 Where Jaguar stands today

The implementation already covers a broad surface. In summary:

AREA	WHAT IS IN PLACE	NOTABLE LIMITS TODAY
Core language	Sized numeric types, <code>auto</code> , <code>const</code> , casts, operators, functions with defaults, named arguments and overloads	Compound assignment is not shown in the reference
Data	Structs, sequential enums, named unions, aliases, forward declarations	Explicit enum initializers are not accepted
Classes	Fields, methods, <code>constr</code> , <code>destr</code> , single inheritance, <code>virtual</code> , <code>override</code> , <code>const</code> methods, access control	No multiple inheritance
Collections	<code>list</code> , <code>map</code> , <code>pair</code> , <code>container</code> , <code>dynamic_list</code> , literals, <code>loop_list</code> , <code>loop_map</code>	Read-only indexing; not valid as globals
Reflection	<code>@register</code> , <code>@exposed</code> , <code>factory:construct</code> , <code>GetMember</code> , <code>SetMember</code> , <code>MemberExists</code>	Opt-in and intentionally limited
Libraries	<code>sys</code> (print, console, files, execute), <code>jcc</code> (math, strings, characters, files, environment)	Math is <code>f64</code> ; integer helpers are <code>i32</code>
C interop	<code>@extern</code> , pointers, <code>nullptr</code> , function pointers, ABI-preserving types, <code>.jah</code>	No variadic functions or C arrays in bindings

AREA	WHAT IS IN PLACE	NOTABLE LIMITS TODAY
Build	JBS 1.0: executables, static and shared libraries, <code>link</code> , <code>define</code> , <code>c89</code> , <code>keep_c</code>	BindGen is not run from a JBS directive
Tooling	<code>jcc.py</code> , <code>jbs.py</code> , <code>jbg.py</code> , source-line diagnostics	Language server and editor integrations are referenced, not yet described here

29.2 Boundaries to protect

Some limits are not gaps. The documentation lists what the language deliberately leaves out, and a healthy roadmap protects that list rather than eroding it:

- general pointer arithmetic;
- `malloc` and `free` as a general API, and `FILE*`;
- `memcpy` and other memory primitives as a general API;
- C++ templates and other complex C++ features;
- lambdas, exceptions and multiple inheritance.

Any proposal that touches this list should be argued as a change to Jaguar's identity, not as a missing feature.

29.3 Candidate work items

These are the concrete limitations that the documentation records, each paired with a proposed direction. The priorities are suggestions.

#	AREA	TODAY	PROPOSED DIRECTION	PRIORITY
1	Enums	Explicit initializers are rejected, and BindGen refuses headers that use them	Accept explicit values, which real C headers rely on	High
2	Build automation	BindGen is a separate step; JBS has no <code>bindgen</code> directive	Add a directive so header conversion is part of the build	High
3	Diagnostics	Parser errors carry lines; some semantic errors use the best available line	Precise locations for every diagnostic, and stable error codes	High
4	Conformance tests	The reference is the specification	Turn every example in the reference into an automated test	High
5	Variadic functions	Refused by BindGen	Define a safe strategy, such as typed wrappers	Medium

#	AREA	TODAY	PROPOSED DIRECTION	PRIORITY
6	C arrays	Refused when the ABI cannot be preserved	Support fixed-size arrays in fields and parameters	Medium
7	Global collections	<code>list</code> , <code>map</code> and others must be local	Allow globals with runtime initialization	Medium
8	Namespace globals	Not supported directly inside a namespace	Support them with mangled names	Medium
9	Toolchain portability	JBS finds a bundled MinGW next to <code>jbs.py</code>	Fall back to a system compiler and configurable paths	Medium
10	Signals	Behavior depends on the current code generation	Specify the semantics, document them and test them	Medium
11	Collection API	The documented surface is small	Document and extend removal, size and lookup helpers	Medium
12	Built-in ergonomics	<code>jcc</code> and <code>sys</code> functions take no named arguments	Allow named arguments for built-ins	Low
13	Loops	<code>loop_list</code> does not work on <code>dynamic_list</code>	Add support	Low
14	Library breadth	Integer helpers are <code>i32</code> ; math is <code>f64</code>	Add <code>i64</code> and <code>f32</code> variants where useful	Low

29.4 Proposed phases

Phase 1: solidify the foundation

Turn the current behavior into something that cannot silently regress. Convert the reference examples into an automated test suite (item 4), improve diagnostics (item 3) and freeze the reference as the specification for version 1. This phase adds no features, and it makes every later phase safer.

Phase 2: complete C interoperability

Interoperability is Jaguar's strongest selling point, so finish it. Accept explicit enum values (item 1), add a BindGen step to JBS (item 2), then tackle arrays and variadic functions (items 5 and 6) and make the toolchain portable (item 9).

Phase 3: broaden the language and library

With a stable base, remove the ergonomic limits: global collections and namespace globals (items 7 and 8), a fuller collection API (item 11), a specified `signal` feature (item 10), and the smaller library and loop gaps (items 12 to 14).

Phase 4: grow the ecosystem

Build on the compiler's source-line tracking. A language server and editor integrations, a formatter, conventions for distributing `.jah` headers with their libraries, a test runner built on `jcc:assert`, and `#line`-based mapping so that debuggers and errors from GCC can point back to Jaguar source.

29.5 How to know a phase is done

Every item in this plan should be closed the same way: the behavior is implemented, a test proves it, the reference documents it, and the example in this book compiles. Documentation and code should move together, because the documentation is how Jaguar describes itself.

MAKE THE PLAN YOUR OWN

This chapter is deliberately editable. Replace the priorities, add real milestones and dates, and drop any item you disagree with. What matters is that each limit in the reference has a named owner and a next step.

APPENDICES

Appendices

Quick references, complete example programs, the implementation checklist and a glossary.

IN THIS PART

- A** Language Quick Reference
- B** Command-Line and JBS Reference
- C** Complete Example Programs
- D** Implementation Checklist
- E** Glossary

Language Quick Reference

A compact reminder of the syntax and the library, for when you know what you want and need to see how it is spelled.

A.1 Types

```
void

int / i32
uint / u32
short / i16
ushort / u16
long / i64
ulong / u64
char / i8
uchar / u8
sbyte / i8
byte / u8
float / f32
double / f64
bool
string

T*
fn(T1, T2, ...) -> R

list<T>
map<K,V>
pair<K,V>
container<T>
dynamic_list
```

TEXT

A.2 Control flow

```
if / else  
while  
for_loop  
break  
continue  
return
```

TEXT

A.3 Classes

```
class  
constr  
destr  
virtual  
override  
this  
$  
%  
const
```

TEXT

A.4 Namespaces

```
namespace  
namespace: function()
```

TEXT

A.5 Attributes

```
@extern  
@register  
@exposed
```

TEXT

A.6 C interop

```
T*
T**
fn(T1, T2, ...) -> R
&
*
nullptr
@extern
```

TEXT

A.7 Preprocessor

```
#define
#undef
#if
#ifdef
#ifndef
#elif
#else
#endif
#pragma
#error
#warning
#line
```

TEXT

`#elseif` is accepted as a Jaguar alias for `#elif`.

`#include` is not supported.

A.8 Collections

```
list.push()
list[index]

map.emplace()
map[key]

pair.first
pair.second

container.get()

dynamic_list.push()
dynamic_list.get()
```

TEXT

```
dynamic_list.type()
dynamic_list.size()
dynamic_list[index]
```

```
loop_list()
loop_map()
```

A.9 System

```
sys:print()
sys:console:set_color()
sys:console:reset_color()
sys:execute()
sys:fs:read()
sys:fs:write()
```

TEXT

A.10 `jcc` standard library

```
jcc:exit()
jcc:abort()
jcc:abs_i32()
jcc:min_i32()
jcc:max_i32()
jcc:clamp_i32()
jcc:random_i32()
jcc:time_ms()
jcc:assert()
```

```
jcc:sqrt()
jcc:pow()
jcc:sin()
jcc:cos()
jcc:tan()
jcc:asin()
jcc:acos()
jcc:atan()
jcc:atan2()
jcc:floor()
jcc:ceil()
jcc:round()
jcc:log()
jcc:log10()
jcc:exp()
jcc:fmod()
```

TEXT

```
jcc:string_length()
jcc:string_equals()
jcc:string_compare()
jcc:string_contains()
jcc:string_starts_with()
jcc:string_ends_with()
jcc:string_concat()
jcc:string_substring()
jcc:string_char_at()
jcc:string_find()
jcc:string_to_upper()
jcc:string_to_lower()

jcc:string_to_i32()
jcc:string_to_i64()
jcc:string_to_f64()
jcc:i32_to_string()
jcc:i64_to_string()
jcc:f64_to_string()

jcc:is_digit()
jcc:is_alpha()
jcc:is_alnum()
jcc:is_space()
jcc:is_upper()
jcc:is_lower()
jcc:to_upper_char()
jcc:to_lower_char()

jcc:file_exists()
jcc:remove_file()
jcc:rename_file()
jcc:env_get()
```

A.11 Syntax summary

```
// comment
/* comment */

#define NAME VALUE

const int x = 10;
auto y = 20;

int add(int a, int b) {
    return a + b;
}

int add(int a, int b);
```

TEXT

```
if (condition) {
} else if (other) {
} else {
}

while {
    break;
    continue;
}

int i = for_loop(0, 10) {
}

struct Vector2 {
    f32 x;
    f32 y;
}

class Player {
    $int health;
    %int protected_value;
    int private_value;

    constr() {
    }

    virtual void update() {
    }

    void update_child() override {
    }

    destr() {
    }
}

namespace game {
    int start() {
        return 0;
    }
}

list<int> values = {1, 2, 3};
map<string, int> scores = {"a", 10};
pair<string, int> result = {"score", 42};
dynamic_list data = {42, "hello", true};
container<int> value => 42;

sys:print("Hello");
jcc:sqrt(25.0);
```

Command-Line and JBS Reference

The three tools at a glance: their command lines, and the complete list of JBS directives.

B.1 `jcc.py`, the compiler

`jcc.py` is the Jaguar compiler.

Basic compilation:

```
python jcc.py main.ja -o game
```

SHELL

Generate C only:

```
python jcc.py main.ja > main.c
```

SHELL

Generate C in C89-compatible block layout:

```
python jcc.py main.ja --c89 > main.c
```

SHELL

With `-o`, JCC invokes the C compiler/linker as part of direct compilation. Without `-o`, it writes generated C to standard output.

The output name supplied to `-o` is an executable/output base name, not a `.c` filename.

B.2 `jbs.py`, the build system

`jbs.py` manages multi-file Jaguar projects.

```
python jbs.py project.jbs
```

SHELL

Select one target:

```
python jbs.py project.jbs --target Main
```

SHELL

Override the configured output directory:

```
python jbs.py project.jbs -o build
```

SHELL

JBS combines `.ja` and `.jah` files, expands `using` imports, calls JCC, then uses the bundled GCC toolchain.

See the **Build System** section for the complete `.jbs` syntax.

B.3 jbg.py, the binding generator

`jbg.py` converts a C header into a Jaguar Header (`.jah`).

Basic usage:

```
python jbg.py --c api.h
```

SHELL

By default this creates:

```
api.jah
```

TEXT

Specify an output path:

```
python jbg.py --c api.h -o include/api.jah
```

SHELL

Write the generated binding to standard output:

```
python jbg.py --c api.h --stdout
```

SHELL

Treat unsupported declarations as a hard failure:

```
python jbg.py --c api.h --strict
```

SHELL

The output file must use the `.jah` extension.

B.4 JBS quick reference

```
version 1.0
out
include
define
c89
keep_c

compile
compile_static
compile_shared
link
```

TEXT

CLI:

```
python jbs.py project.jbs
python jbs.py project.jbs -o build
python jbs.py project.jbs --target Main
```

TEXT

JBS expands `using` imports, uses JCC for Jaguar → C, and then uses the bundled GCC toolchain for final compilation and linking.

Complete Example Programs

Two complete programs that use several features together. Read them, compile them and take them apart.

C.1 A program that uses many features

Here is a small program using several features of the language:

```
#define MAX_HEALTH 100

@register
class Player {
    @exposed
    $int health = MAX_HEALTH;

    constr(int initial_health = MAX_HEALTH) {
        health = initial_health;
    }

    void damage(int amount) {
        health = health - amount;
    }

    bool alive() const {
        return health > 0;
    }
}

int add(int a, int b = 0) {
    return a + b;
}

void main(string param) {
    Player player = Player(100);

    list<int> values = {10, 20, 30};

    auto item = loop_list(values) {
        sys:print(item);
    }
}
```

JAGUAR

```

player.damage(25);

sys:print(player.health);

if (player.alive()) {
    sys:print("Player alive");
}

f64 distance = jcc:sqrt(3.0 * 3.0 + 4.0 * 4.0);
sys:print(distance);

string text = jcc:i32_to_string(add(10, b = 20));
sys:print(text);
}

```

C.2 A small game program

```

class Player {
    $f32 x = 0.0;
    $f32 y = 0.0;
    $int health = 100;

    void move(f32 dx, f32 dy) {
        x = x + dx;
        y = y + dy;
    }

    void damage(int amount) {
        health = health - amount;
    }

    bool alive() const {
        return health > 0;
    }
}

void main(string param) {
    Player player = Player();

    int i = for_loop(0, 4) {
        player.move(1.0, 0.5);
        sys:print(i);
    }

    player.damage(20);

    if (player.alive()) {
        sys:print("Player is alive");
    }
}

```

JAGUAR

```
    }  
  
    sys:print(player.x);  
    sys:print(player.y);  
}
```

Implementation Checklist

What the current implementation includes, gathered in one list, and how to read the status of a feature.

D.1 How to read the status of a feature

This documentation matches the behavior currently present in `jcc.py`.

Points to consider as particularly specific to the current implementation are:

- the conditionless form of `while`;
- `for_loop` with an inclusive final bound;
- collections without C++ templates;
- the ban on modifying a collection during `loop_list` / `loop_map`;
- `container<T>`;
- `dynamic_list`;
- the `@register` / `@exposed` / `factory:construct` system;
- the `sys:*` functions;
- the `jcc:*` library;
- the C89 generation strategy;
- name mangling and overload resolution.

For any feature not described here, refer to the actual behavior of `jcc.py` rather than assuming it is inherited from C or C++.

D.2 Current syntax checklist

The current implementation includes, among other features:

TEXT

Types:

- void
- int / i32
- uint / u32
- short / i16
- ushort / u16
- long / i64
- ulong / u64
- char / i8
- uchar / u8
- sbyte / i8
- byte / u8
- float / f32
- double / f64
- bool
- string
- T*
- fn(T1, T2, ...) -> R
- list<T>
- map<K,V>
- pair<K,V>
- container<T>
- dynamic_list

Declarations:

- variable
- global variable
- @extern global
- function
- prototype
- struct
- union
- enum
- class
- forward declaration
- using type alias

Functions:

- default parameters
- named arguments
- overloads
- const methods
- virtual
- override
- callbacks/function pointers

Control:

- if / else if / else
- while
- for_loop
- loop_list

```
loop_map  
break  
continue  
return
```

Expressions:

```
arithmetic  
comparisons  
logical operators  
C-style casts  
address-of  
dereference  
member access  
indexing  
calls  
new  
nullptr  
list/map/pair/dynamic_list literals
```

Modules:

```
using file  
using namespace  
using symbol  
using symbol as alias  
namespace
```

Reflection:

```
@register  
@exposed  
factory:construct  
MemberExists  
GetMember  
SetMember
```

C interoperability:

```
@extern  
pointers  
pointer-to-pointer  
function pointers  
&  
*  
nullptr  
.jah bindings
```

Build:

```
JBS 1.0  
compile  
compile_static  
compile_shared  
link  
include  
define
```


c89
keep_c

Tools:

jcc.py
jbs.py
jbg.py

Glossary

The vocabulary of Jaguar and its toolchain, in alphabetical order.

TERM	MEANING
ABI	The binary conventions for how types are laid out and functions are called. Jaguar preserves the C ABI when it talks to C.
AST	The abstract syntax tree that the parser builds from the token stream.
Attribute	An annotation that begins with <code>@</code> , such as <code>@extern</code> , <code>@register</code> or <code>@exposed</code> .
BindGen	The tool (<code>jbg.py</code>) that converts a C header into a Jaguar header.
C89 mode	A code generation mode, enabled with <code>--c89</code> or the JBS <code>c89</code> directive, that produces C89-compatible block layout.
Code generator	The compiler stage that writes C from the resolved program.
Compilation unit	What the compiler processes at once. With JBS, all listed files and their imports are combined into one.
Container	A <code>container<T></code> value, whose data is owned and eventually destroyed by the container.
Function-pointer type	A C-compatible type written <code>fn(T1, T2) -> R</code> .
GCC	The C compiler that turns the generated C into an executable.
Jaguar header (<code>.jah</code>)	A Jaguar interface file, meant to be reused with <code>using</code> .
JaguarCC (<code>jcc.py</code>)	The Jaguar compiler.
JBS (<code>jbs.py</code>)	The Jaguar Build System, which builds projects from <code>.jbs</code> files.
Lexer	The compiler stage that turns source text into tokens.
Name mangling	Adapting a name for C so that namespaces and overloads do not collide, for example <code>math_add</code> .
Namespace	A named scope for functions, structs, classes and nested namespaces, with <code>:</code> as the separator.
Overload	One of several functions that share a name and differ in parameters.
Parser	The compiler stage that builds a syntax tree from tokens.

TERM	MEANING
Prototype	A function declaration without a body.
Reflection	Creating objects by name and reading or writing their exposed members at runtime.
Resolver	The compiler stage that resolves names, overloads, conversions and access rules.
Runtime	The small support code generated with a program, which manages strings, classes, containers, lists, maps and dynamic lists.
Signal	The <code>signal:</code> syntax that attaches a block to a variable, to react to changes.
<code>using</code>	The keyword for importing a file, opening a namespace, importing one symbol, or declaring a type alias.
<code>_jBool</code>	The runtime's own boolean type used in generated C instead of <code>_Bool</code> .

ALPHABETICAL

Index

\$

\$ 43

%

% 43

=

=> 76

A

A compiler in four stages 130
 A complete example 84
 A complete reflection example 59
 A Development Plan 134
 A practical C API example 77
 A program that uses many features 148
 A small game program 149
 A Tour of Jaguar 10
 ABI-preserving numeric types 75
 Access control 43
 Anonymous aggregate types 82
 Attributes 59, 140
 auto 24, 76, 126

B

Boundaries to protect 135
 break 36
 Building projects 14

C

.c 128
 C interop 141
 C Interoperability 72
 C strings 74
 C strings and ABI preservation 82
 C to Jaguar cheat sheet 77
 c89 88
 Candidate work items 135
 Casts 26
 char* 126

Character functions 68
 char_at() 30
 Classes 12, 140
 Classes and Objects 42
 Collections 12, 141
 Collections and Object Ownership 47
 Command-Line and JBS Reference 145
 Comments 18
 compile 89
 compile_shared 90
 compile_static 90
 Compiling Your First Program 6
 Complete Example Programs 148
 concat() 29
 Conditions 35
 Console colors 63
 const 23, 46, 76
 constr 44
 Constructing an empty string 30
 Containers 52
 contains() 29
 continue 36
 Control flow 11, 140
 Control Flow and Scope 35
 Core types 22
 Current syntax checklist 151

D

Declaring a class 42
 Declaring and calling functions 31
 Default parameters 31
 define 88
 Design philosophy 3
 destr 45
 Diagnostics and source locations 97
 Dynamic lists 49

E

else 35
 else if 35
 empty() 29

ends_with() 29
 Enumerations 39
 env_get 69
 equals() 29
 @extern 34, 72, 76, 128

F

factory:construct 58
 File system 63
 Files and environment 69
 file_exists 69
 Forward declarations 41
 for_loop 37
 From one compiler to a toolchain 132
 From source to executable 95
 Full project example 92
 Function-pointer types 74
 Functions 10, 31

G

General syntax 17
 Generated C and compiler compatibility 77
 GetMember 58
 Global variables 24
 Glossary 155

H

Hello, Jaguar 6
 How a call is resolved 33
 How BindGen works 80
 How C interoperability grew up 131
 How to know a phase is done 137
 How to read the status of a feature 151

I

if 35
 Implementation Checklist 151
 Implicit numeric conversion in calls 26
 Importing a complete Jaguar file 56
 Importing a namespace 55
 Importing one symbol, with an alias 55
 #include 128
 include 87
 Indexing 51
 Inheritance 45
 Inheritance and virtual methods 13
 Invalid-header recovery 83

J

Jaguar Headers and BindGen 79
 .jah 79, 80
 jbg.py 146
 JBS command line 91
 JBS quick reference 147
 jbs.py 145
 jcc 65, 142
 jcc.py 145
 jcc:abort 65
 jcc:abs_i32 66
 jcc:acos 67
 jcc:asin 67
 jcc:assert 66
 jcc:atan 67
 jcc:atan2 67
 jcc:ceil 67
 jcc:clamp_i32 66
 jcc:cos 67
 jcc:env_get 69
 jcc:exit 65
 jcc:exp 67
 jcc:f64_to_string 68
 jcc:file_exists 69
 jcc:floor 67
 jcc:fmod 67
 jcc:i32_to_string 68
 jcc:i64_to_string 68
 jcc:is_alnum 69
 jcc:is_alpha 69
 jcc:is_digit 69
 jcc:is_lower 69
 jcc:is_space 69
 jcc:is_upper 69
 jcc:log 67
 jcc:log10 67
 jcc:max_i32 66
 jcc:min_i32 66
 jcc:pow 67
 jcc:random_i32 66
 jcc:remove_file 69
 jcc:rename_file 69
 jcc:round 67
 jcc:sin 67
 jcc:sqrt 67
 jcc:string_char_at 68
 jcc:string_compare 68
 jcc:string_concat 68
 jcc:string_contains 68
 jcc:string_ends_with 68
 jcc:string_equals 68

jcc:string_find	68
jcc:string_length	68
jcc:string_starts_with	68
jcc:string_substring	68
jcc:string_to_f64	68
jcc:string_to_i32	68
jcc:string_to_i64	68
jcc:string_to_lower	68
jcc:string_to_upper	68
jcc:tan	67
jcc:time_ms	66
jcc:to_lower_char	69
jcc:to_upper_char	69

K

keep_c	89
--------	----

L

Language Quick Reference	139
length()	29
link	90
Lists	47
Literals	18
loop_list	51
loop_map	51

M

main	33
Maps	48
Math	66
Meet Jaguar	2
MemberExists	58
Memory management	53
Methods	43
Minimal project	86

N

Name mangling	96
Named arguments	32
Namespaces	13, 54, 140
Namespaces and Modules	54
new	52
NULL	125
nullptr	74, 125

O

One unit, many files	97
Operators	25
out	87

Overloading	33
override	127

P

Pairs	49
Pointer types	73
Preprocessor	141
Printing	62
Process control and integer helpers	65
Program Structure and Lexical Elements	17
Proposed phases	136
Prototypes	32

R

Recommended project organization	94
Reflection	15
Reflection, Attributes and Signals	57
remove_file	69
rename_file	69
Reserved libc names	69
Returning values	34
Running a program	63

S

SetMember	59
Signals: reacting to a variable change	60
Source files	17
starts_with()	29
string	28, 126
String conversions	68
String functions	67
Strings	28
Structs	39
Structs, Enums, Unions and Aliases	39
substring()	29
Supported C declarations	81
Syntax summary	143
sys	62, 64
sys:console:reset_color	63, 64
sys:console:set_color	63, 64
sys:execute	64
sys:fs:read	64
sys:fs:write	64
sys:print	62, 64
System	142

T

Talking to C	14
--target	91

The collection types	47
The JBS Build System	86
The jcc Library	65
The preprocessor	19
The principles behind the choices	132
The signature decisions	131
The standard library	14
The starting idea: a language that compiles to C . . .	130
The Story of Jaguar	130
The sys Library	62
The toolchain at a glance	4
this	44
Three ways to run the compiler	7
Tool responsibilities	97
Toolchain	92
Top-level expressions	20
to_lower()	30
to_upper()	30
Types	139
Types, Variables and Operators	22

U

Under the Hood: How JaguarCC Works	95
Unions	40
Unsupported C constructs	83
using	41, 92

V

Variable scope	37
Variables	23
Variables and types	10
version	87
Virtual methods	46
void	31

W

What ends up in the C	96
What is Jaguar?	2
What Jaguar deliberately leaves out	3
What just happened?	8
What you need	6
Where collections can and cannot be used	53
Where Jaguar stands today	134
while	36, 124

{

{	50
-------------	----



T H E E N D

*Thank you for reading.
Keep the compiler close and the documentation closer.*

JAGUAR

Compiled. Explicit. Interoperable.